

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 434 — Past Questions & Materials
by Renance

Course: CSC 434
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 10
Levels: 400

This study pack was compiled and restructured by Renance for CSC 434 students at Federal University of Petroleum Resources, Effurun. It brings together 10 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

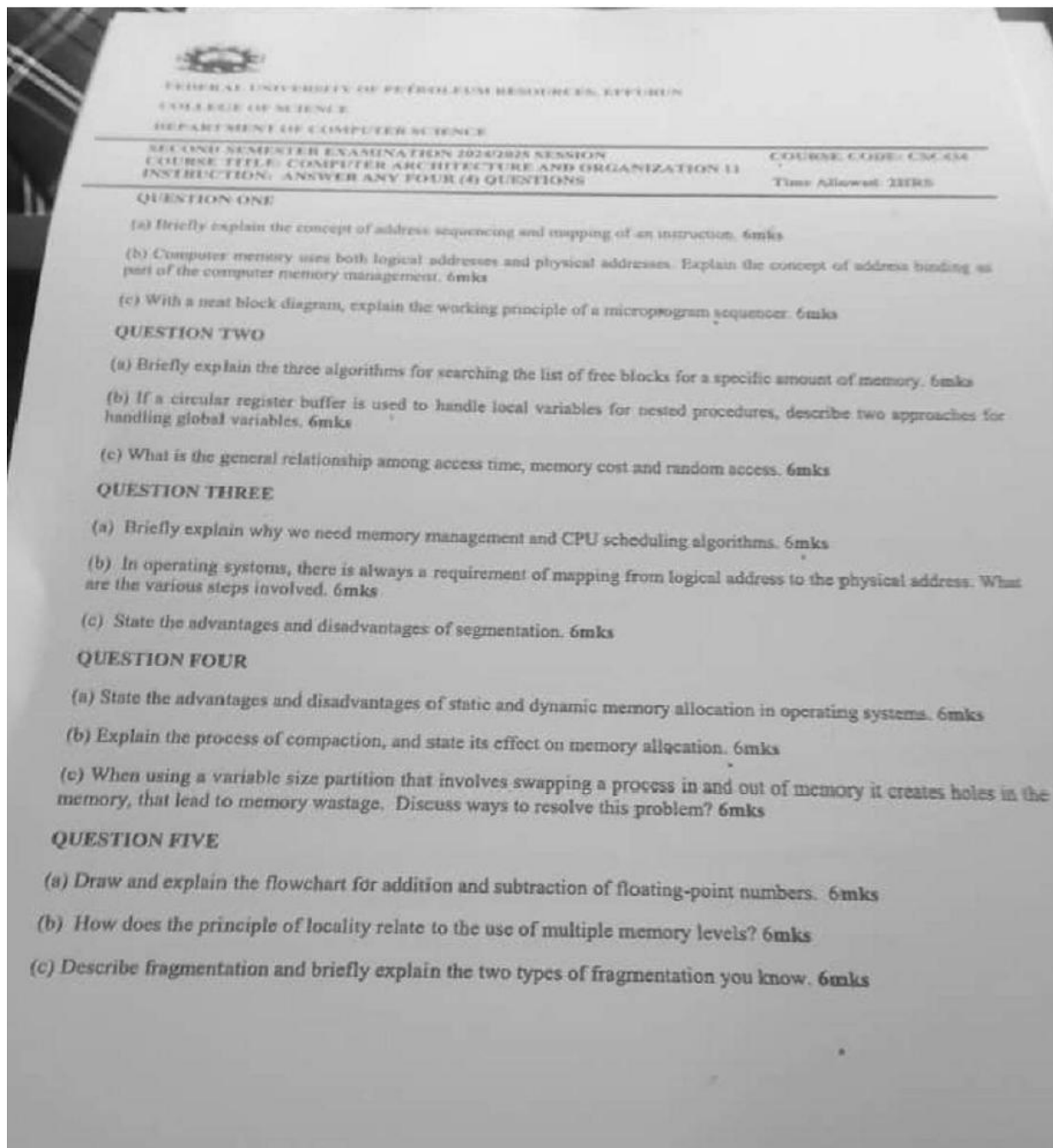
Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 10 — CSC 434 PQ 2025

Level 400 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



Paper 2 of 10 — CSC 434 TEST 2025

Level 400 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.

CSC434 (COMPUTER ARCHITECTURE AND ORGANIZATION II) TEST

1. State the advantages and disadvantages of static and dynamic memory allocation in operating systems.
2. When using a variable size partition that involves swapping a process in and out of memory it creates holes in the memory, that lead to memory wastage. Discuss ways to resolve this problem?
3. Explain the process of compaction and state its effect on memory allocation

Paper 3 of 10 — CSC 434 PQ 2023

Level 400 • Second Semester • PastQuestion

FEDERAL

UNIVERSITY OF PETROLEUM RESOURCES

melt

COLLEGE OF SCIENCE

j

~ DEPARTMENT OF COMPUTER SCIENCE

FIRST

Course EXAMINATION 2022/2023 SESSION COURSE

TITLE COMPUTER ARCHITECTURE AND ORGANIZATION 11

INSTRUCTION: ANSWER ANY FOUR (4) QUESTIONS

QUESTION ONE

& 13) While processor power has raced to breakneck speed, other critical components

have not kept up. The result is a performance imbalance: architecture

cannot compensate for the mismatch among the capabilities of the various

will a architect attack this problem?

(a) Computer memory uses both logical addresses and physical addresses. Explain the concept of

binding is part of the computer memory management, 6 marks

(b) Define memory partitioning and briefly explain the different ways in which memory is divided

Sections

QUESTION TWO

(a) The power of the processor (number of instructions per second) is a key factor in determining the

contemporary processor to achieve

(b) If register buffer is used to handle local variables for procedures, describe the various approaches for handling global

variables in which a OM pipeline can deal with conditional branches

QUESTION THREE

(a) ROMs have been organized as only one bit per chip. whereas RAMs are usually organized as 512 bits per chip. Discuss

the different locations in main memory, what is the implementation

dynamic

Environment? 5 marks

Describe the major types of Operating systems (OS) scheduling algorithms

QUESTION FOUR

(a) Briefly explain:

(i) compile time (ii) load time (iii) execution time, Services

are

(0) State the advantages and disadvantages of static and dynamic memory allocation in Operating systems. 6mks

• solves swapping @ process in and out of memory it eradicates %
 • that lead to memory wastage. Discuss ways to resolve this problem? 5mks

(c) When using 3 variable sized partitions that leave holes in the memory, then
 w QUESTION Five

(a) For a set-associative cache, a main memory address is viewed as consisting of three fields: List and define the three fields. 6m

(9) How does the principle of locality relate to the use of multiple memory levels? 6mks

(c) Describe fragmentation and briefly explain the two types of fragmentation you know. 6mks

pa sl

Paper 4 of 10 — CSC 434 PQ 2024

Level 400 • Second Semester • PastQuestion

SSS

gee FEDE

ce; RAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

x COLLEGE OF SCIENCE

—— DEPARTMENT OF COMPUTER SCIENCE

COURSE cone. eae SEMESTER EXAMINATION 2023/2024 SESSION

INSTRUCTION, 4 COURSE TITLE: COMPUTER ARCHITECTURE AND ORGANIZATION I1

+ ANSWER ANY FOUR (4) QUESTIONS Time Allowed: 2HRS 30 MINS

QUESTION one

(8) Whar;

tb} atis the general relationship among access time, memory cost and random access, (5*/amks]

Co

sas) Memory uses both logical addresses and physical addresses, Explain the concept of

4

@ o Gress binding as part of the computer memory management. {6mks}.

éfine memory partitioning and-briefly explain the different ways In which memory can be

Partitioned, lexi

QUESTION Two

{a)

Sriefly explain the characterization of the Reduced Instruction Set Architecture. (5*/amks].

(b) Ifa circular register buffer ls used to handle local variables for nested procedures, describe two approaches for handling global variables, [6mks].

(c) Briefly explain the two basic tasks that 2 control unit performs. [6mks].

QUESTION THREE

(8) What are the differences among Sequential access, Direct access and Radom access, [6mks).

(b) if a process is dynamically assigned to different locations in main memory, what is the Implication for the addressing mechanism? [6mks). :

{c) Explain the distinction between the written sequence and the time se Ace af an instruction, [5*/2mks]. ra

Sa

QUESTION FOUR

{a) Provide a typical list of the inputs and outputs of a control unit. [6mks).

(b) What are the differences between dynamic partitioning and fixed partitioning? [6mks]. -

(c) When using a variable size partition that involves swapping a process in and out of memory, it creates holes in the memory, that lead ta memory wastage. Discuss ways to resolve this

problem? [5*/2mks].

QUESTION FIVE

(a) Why do we need memory management and CPU scheduling algorithms for multiuser F}

systems? [6mks].

(b) How does the principle

(ce) Describe fragmentation en

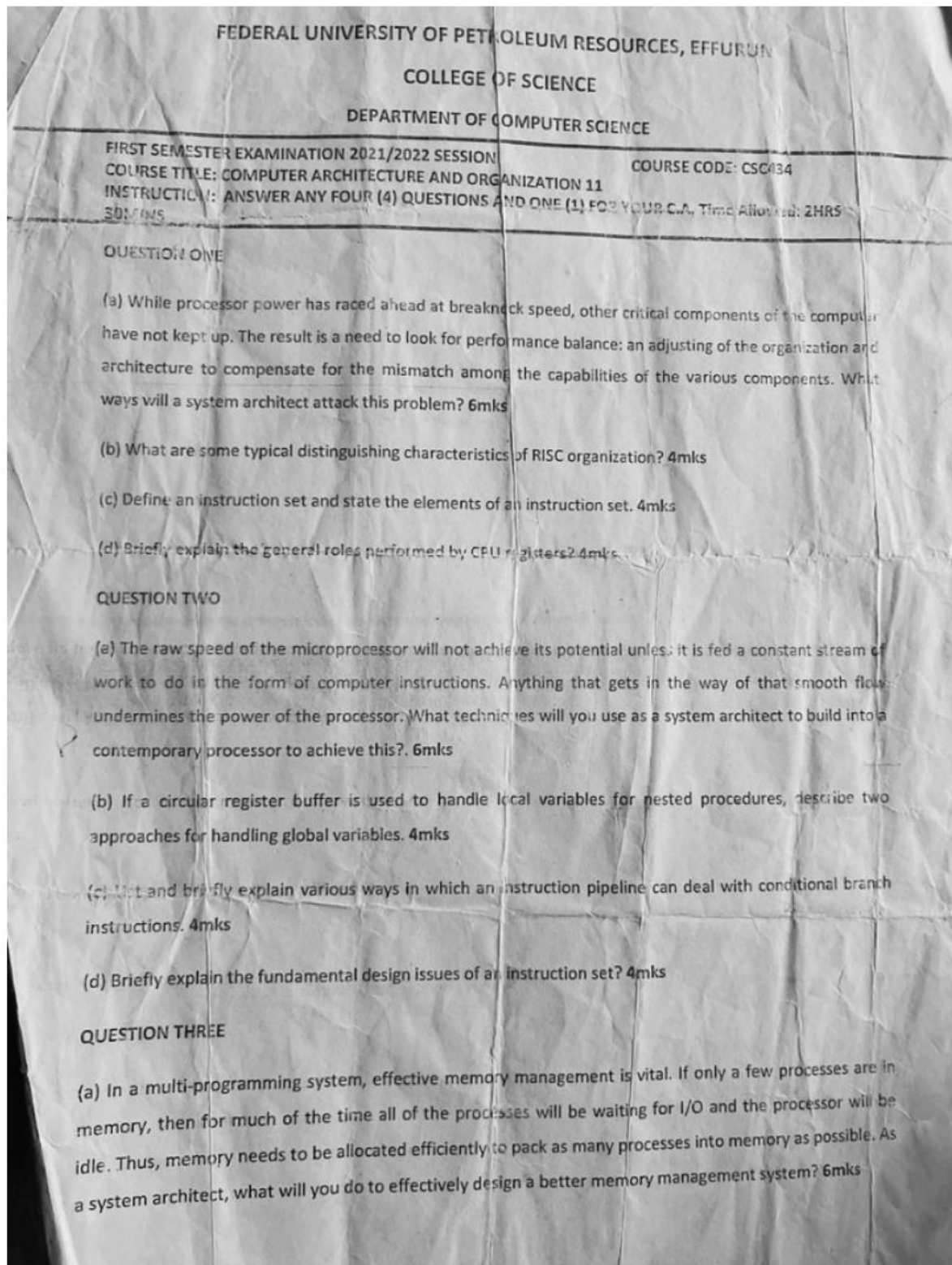
of locality relate to the use of multiple memory levels? [5*/2mks].

d briefly explain the two types of fragmentation you know, [6mks].

Paper 5 of 10 — CSC 434 PQ

Level 400 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



(b) If a process is dynamically assigned to different locations in main memory, what is the implication for the addressing mechanism? 4mks

(c) List and briefly define the major types of Operating System (OS) scheduling. 4mks

(d) Define a cache memory and give reasons why we cannot have a big volume of cache memory. 4mks

QUESTION FOUR

(a) A computer is a complex system; contemporary computer contains millions of elementary electronic components. How, then, can one clearly describe them? 5mks

(b) Justify the assertion that a 32-bit instruction is probably less than twice as useful as a 16-bit instruction. 3mks

(c) Outline a three-step process that leads to a characterization of the control unit. 5mks

(d) When using a variable-size partition that involves swapping a process in and out of memory it creates holes in the memory, that lead to memory wastage. Discuss ways to resolve this problem? 5mks

QUESTION FIVE

(a) For a direct-mapped cache, a main memory address is viewed as consisting of three fields. List and define the three fields. 4mks

(b) Distinguish between spatial locality and temporal locality? 4mks

(c) Briefly explain the differences between sequential access of data and random access. 6mks

(d) Is it necessary for all of the pages of a process to be in main memory while the process is executing? Discuss. 4mks

QUESTION SIX

(a) Briefly explain the concept of stack and its implementation. 4mks

(b) Explain the concept of Page Replacement Algorithm in memory management. 5mks

(c) How does the principle of locality of reference relate to the use of multiple memory levels? 4mks

(d) A set of 4 associative cache consists of 64 lines, or slots, divided into four-line sets. Main memory contains 4K blocks of 128 words each. Show the format of main memory addresses. 5mks

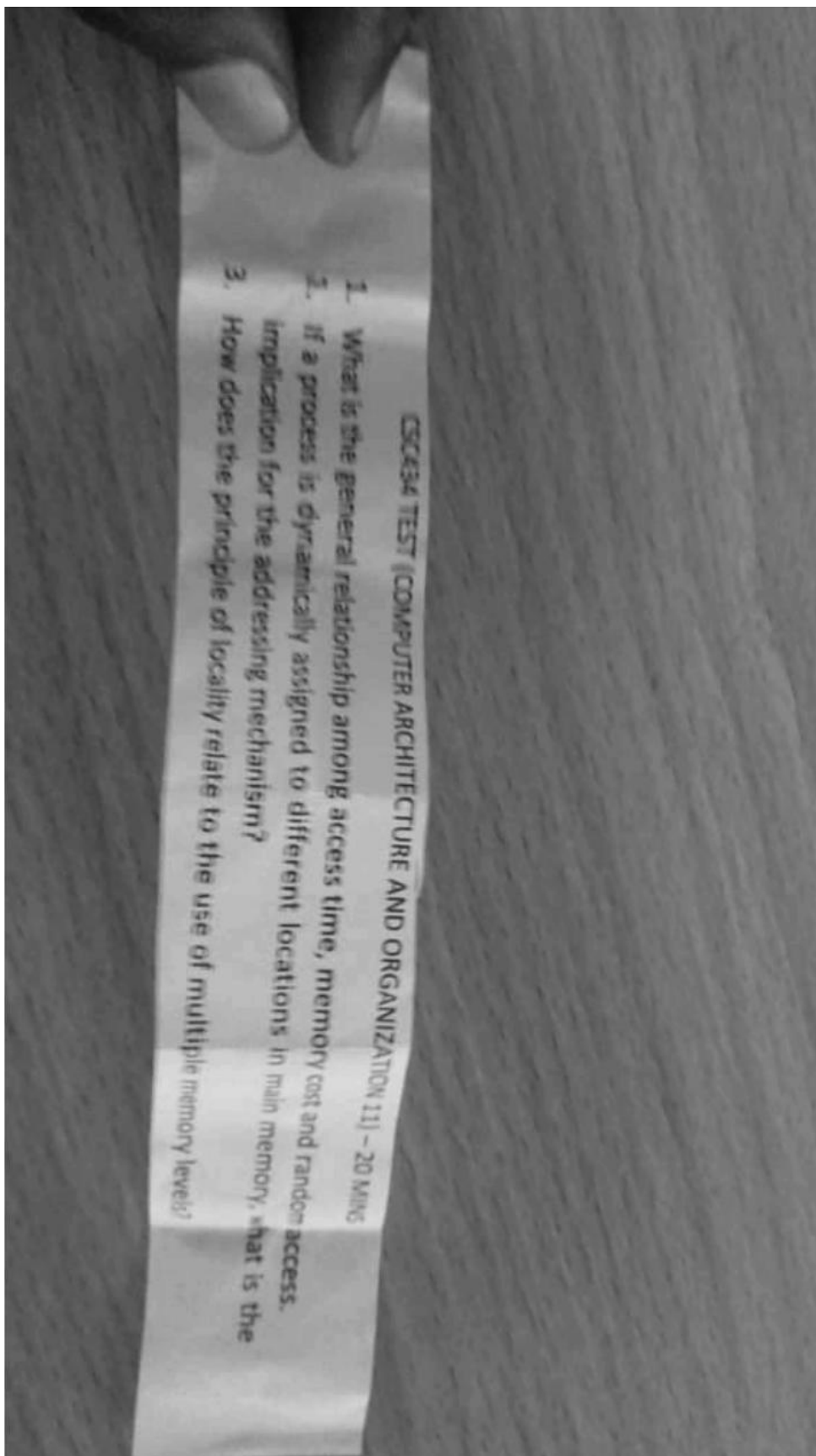
Mode: med

Page 4 of 4
Page 4 of 4 17/11/2022

Paper 6 of 10 — CSC 434 test question

Level 400 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



Paper 7 of 10 — CSC 434

Level 400 • Second Semester • PastQuestion

[c] List and briefly define the major types of Operating system (05) each.

Memory allocation in Operating systems \

(b) State the advantages and disadvantages of static and dynamic memory allocation \

(c) When using 3 variable size Partition that involves swapping a process in and out of memory, it creates %

holes in the memory, that lead to memory wastage. Discuss ways to resolve this problem? 5 marks |

(a) For a set-associative cache, a main memory address is viewed as consisting of three fields, List and define the three fields ; 6 marks

(b) How does the principle of locality relate to the use of multiple memory levels?, 6 marks

(c) Describe fragmentation and briefly explain the two types of fragmentation you know, 6 marks

1. Internal

Fragmentation

2. External

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

Fragmentation

t e\

OT OR Ris | .

: ~/

~\ F

7

will not achieve its potential unless. it is fed a constant stream

'instructions. file that gets H the way of that smooth file

at technique: test will you use as @ system architect to build into

cr far = buffer area to keep the data for

in handling global environment

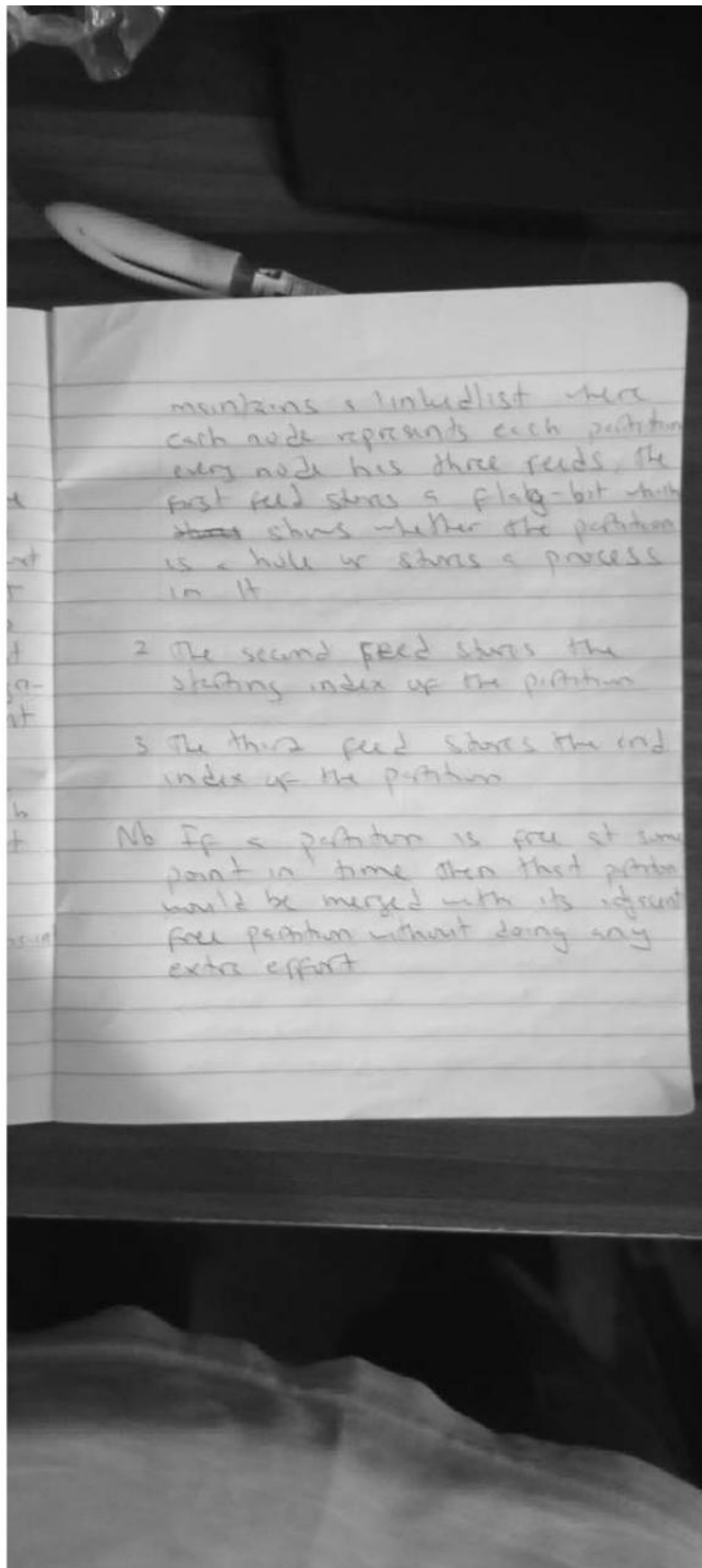
permeant is vital. if only a few instances are in

will be waiting for the data to be read

Paper 8 of 10 — CSC 434 CLASS NOTE 2025,PART2

Level 400 • Second Semester • Handout

This source paper is a scanned document; its pages are reproduced below.

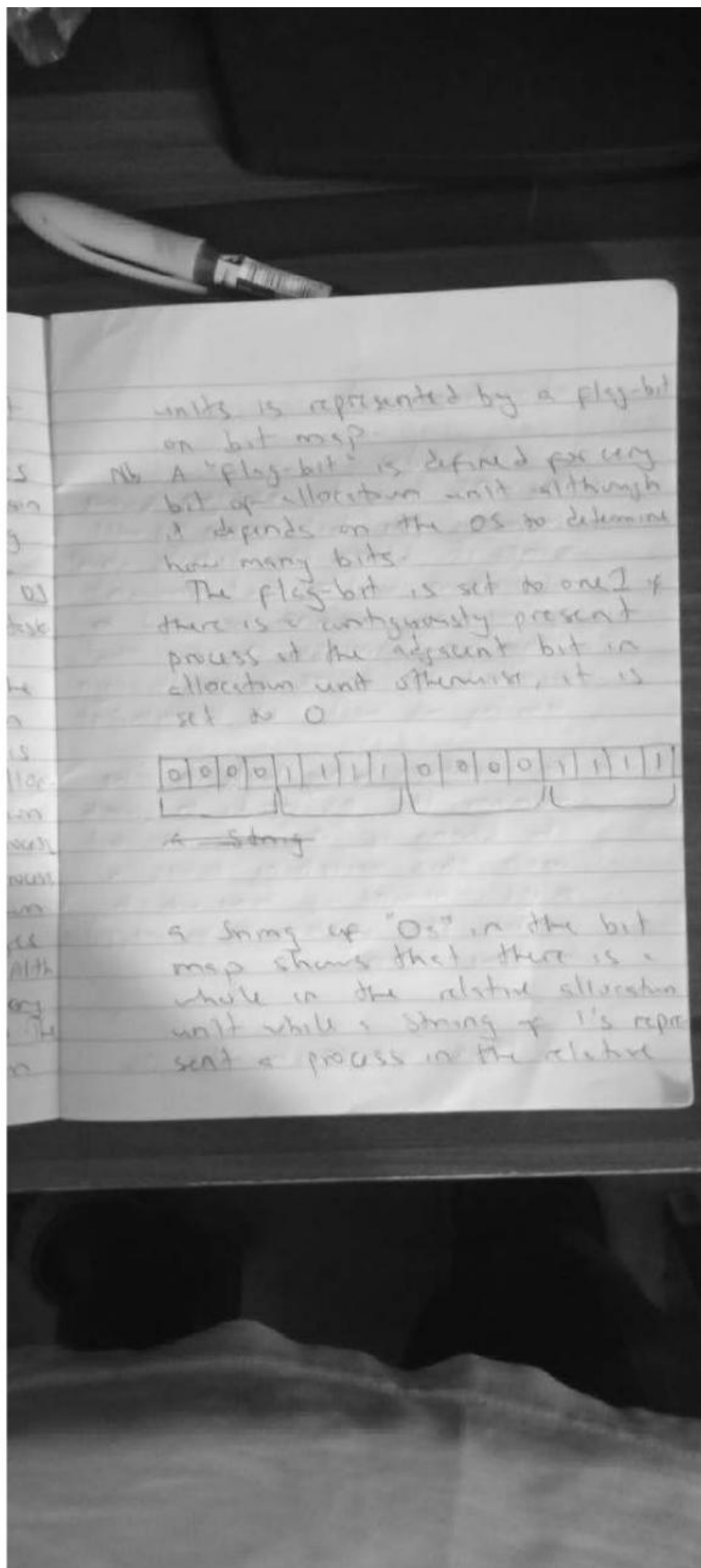


allocation unit

Disadv

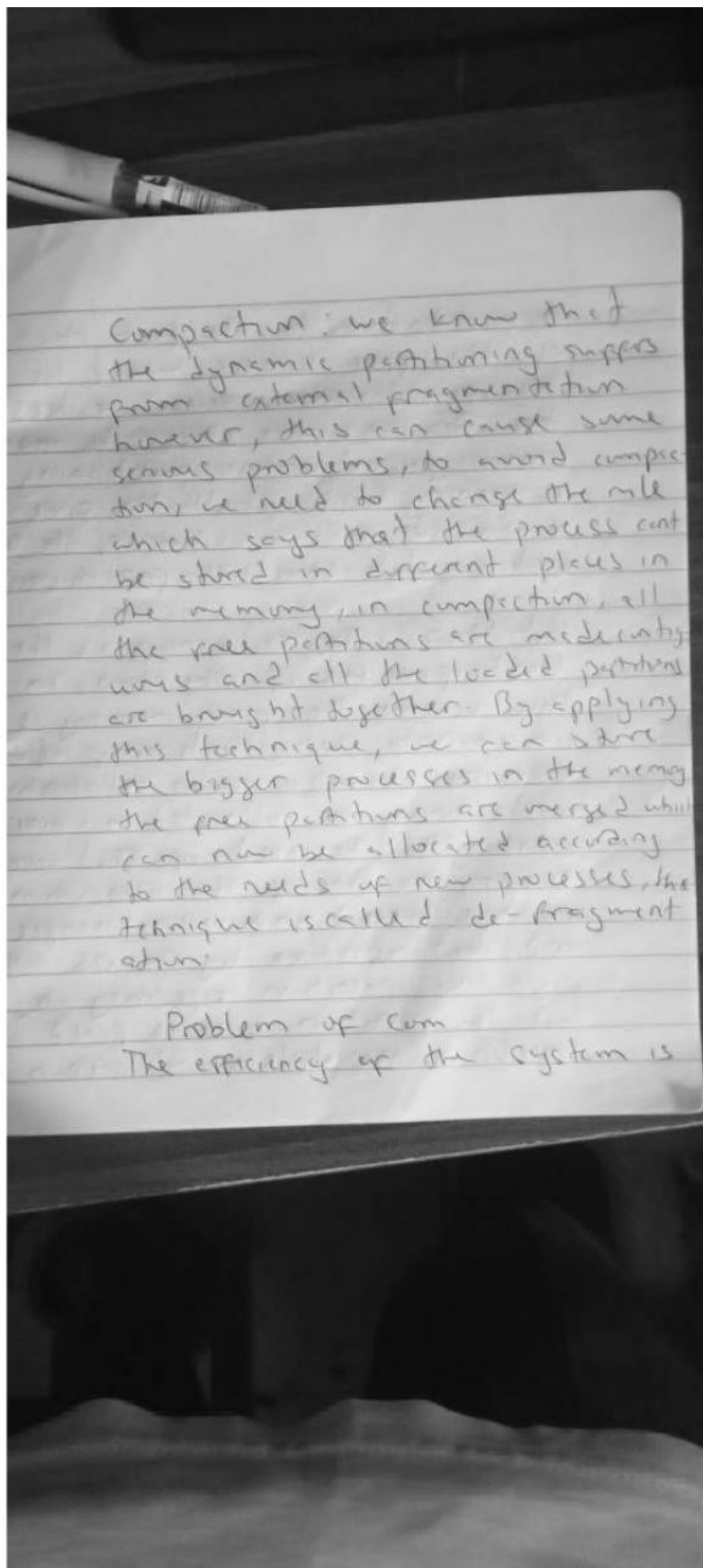
1. The OS has to assign some memory for bit map as well since it stores the details about allocation units that amount of memory cannot be used to load any process therefore that decreases the degree of multiprogramming as well as throughput.
2. To identify any hole in the memory, the OS needs to search the string of "0s" in the bit map, this searching takes a huge amount of time which makes the system to be inefficient to some extent.

In this approach, the OS



decreased due to the fact that all the free spaces would be transferred from several places ~~to~~ to the free space. The main concern for dynamic partitioning is keeping track of all the free & allocated partitions however, the OS uses two data structures for this task.

Bit Map: The bit map is the least famous of the two, in this scheme, the main memory is divided into the collection of allocation units, one or more allocation units may be allocated to a process according to the need of that process however, the size of the allocation unit is fixed and never changes and that is done by the OS. Although the partition size may vary but the partition size is fixed to process or the whole on allocation.



Compaction: we know that the dynamic partitioning suffers from external fragmentation however, this can cause some serious problems, to avoid compaction, we need to change the rule which says that the process can't be stored in different places in the memory, in compaction, all the free partitions are made contiguous and all the loaded partitions are brought together. By applying this technique, we can store the bigger processes in the memory, the free partitions are merged which can now be allocated according to the needs of new processes, the technique is called de-fragmentation.

Problem of compaction
The efficiency of the system is

Disadvantages of Dynamic

1. External Fragmentation

The rule says that the process must be contiguously present in the main memory to get executed. We need to change this rule to avoid external fragmentation.

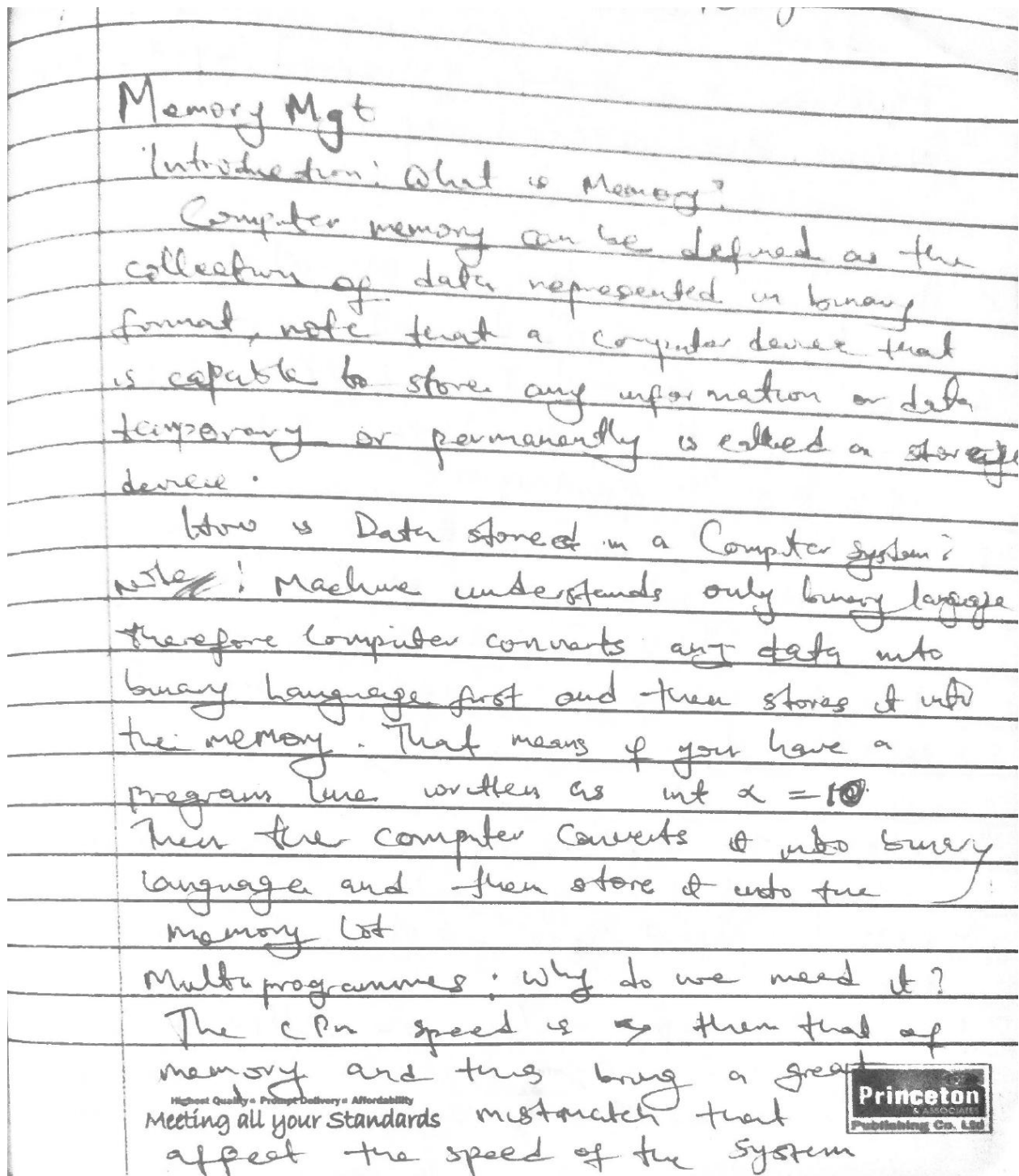
Complex Memory Allocation

In fixed partitioning, the list of partitions is made once and will never change but in dynamic partitioning, allocation and de-allocation is very complex since the partition size would be varied any time its assignment to a new process takes place. The OS has to keep track of all the partitions.

Paper 9 of 10 — CSC 434 CLASS NOTE Part 1 2025

Level 400 • Second Semester • Handout

This source paper is a scanned document; its pages are reproduced below.



making it slow. There must be mechanism that contemporary computers must adhere in terms of memory management to equalize the processing speed of the CPU. Hence the concept of multi programming.

CSC 434

The size

If the size of the main memory is larger than the CPU can load more processes

In the main memory at the same time and therefore will increase the degree of multiprogramming as well as the CPU utilization.

Example: Given a process size of 4mb and the main memory size of 4mb, this process can only reside in the main memory at any time, if the time for which the process does its I/O-operation is P , then CPU utilization $= 1 - P$

CPU utilization is a metric that grows

Highest Quality • Prompt Delivery • Affordability
Meeting all your Standards

Princeton
& ASSOCIATES
Publishing Co. Ltd

the percentage of time spent actually processing instructions or performing useful work for a system
lets assume P is 70%

$$\text{CPU utilization} = 30\%$$

increase memory size to 8mb

what is CPU utilization

$$\text{CPU utilization} = 1 - P/N$$

$$= 51\%$$

we can say

The CPU will be increased if the memory size is

Fixed partitioning.

This is one of the earliest and simplest techniques which can be used to load more than 1 processes into the main memory. It is also

known as contiguous memory allocation.

In this technique the main memory is divided into partitions of equal or different sizes.

Note the operating system always reside in the first partition while the other partition can be used

to store user process and

Meeting all your standards processes assigned in memory in a contiguous way



Q's

- The partition cannot be alive
- The process must be contiguously present in a partition

-

What are issues of fixed Partition

① Internal fragmentation

If the size of the process is lesser than the total size of the partition then some spaces of the partition gets wasted and remain unknown

② External fragmentation

The total unused space of various partition cannot be used to know the processes even though there is space available, but not in a contiguous form

③ Limitation on the size of the process

If the process size is larger than the size of the max size partition then that process cannot be loaded

④ Degree of multiprogramming is less, this is because the size of the partition cannot be vary according to the size of the spaces

Highest Quality, Prompt Delivery, Affordability
Meeting all your standards



Dynamic partitioning tries to overcome the problem caused by fixed partition. In this technique it is declared at the time of process.

In dynamic the size of the partition is equal to the size of the process so $\therefore$ internal fragmentation is avoided. Advantages

- No internal fragmentation
- No limitation to size of processes
- The Degree of multiprogramming is dynamic

Highest Quality • Prompt Delivery • Affordability
Meeting all your Standards

Princeton
& ASSOCIATES
Publishing Co. Ltd

Paper 10 of 10 — CSC 434

Level 400 • Second Semester • Handout

UNIT - 8

Reduced Instruction Set Computer Architecture

Lesson Structure

8.0 Objectives

8.1 Introduction

8.2 Introduction to RISC

8.2.1 Importance of RISC Processors

8.2.2 Reasons for Increased Complexity

8.2.3 High Level Language Program Characteristics

8.3 RISC Architecture

8.4 The Use of Large Register File

8.5 Comments on RISC

8.6 RISC Pipelining

8.7 Summary

8.8 Questions

8.9 Suggested Readings

8.0 Objectives

After going through this Unit you should be able to :

- define and describe the reasons for developing RISC;
- describe the design principles of RISC;
- describe why large register file are important for fast processing;
- define and describe RISC pipelining and the optimisation in RISC pipelining.

Reduced Instruction Set Computer Architecture

8.1 Introduction

We have already discussed the instruction set, register organization and pipelining, and control unit organization in the previous unit. In the early years a large instruction set, a large number of addressing modes and about 16-32 registers were used. But it was required to make the instruction set simple and less. This led to the development of a new type of computers called Reduced Instruction Set Computer (RISC). In this unit, we will discuss about the RISC machines. Our emphasis will be on discussing the basic principles of RISC and its pipelining along with a comparative analysis of RISC and CISC architecture.

6.2 Introduction to RISC

The RISC stands for Reduced Introduction Set Computer. To put simply, RISC is a microprocessor which runs using a pipelining architecture to improve the

performance of a processor. Generally speaking this means faster machine, mostly by improving MIPS (which stands for millions of instructions per sec, meaning higher MIPS are better). It is important to note that improvement of MIPS isn't always result in faster machine, as measurement of MIPS alone isn't good enough to measure the processor. Some well known microprocessors such as SUN Microsystems' SPARC microprocessor or DEC's Alpha microchips uses the RISC concept to develop their microprocessors.

Few more improvements by using RISC processors besides improving MIPS are :

1. A new microprocessor can be developed and tested more quickly if one of its aims is to be less complicated.
2. Operating system and application programmers who use the microprocessor's instructions will find it easier to develop code with a smaller instruction set.
3. The simplicity of RISC allows more freedom to choose how to use the space on a microprocessor.
4. Higher-level language compilers produce more efficient code than formerly because they have always tended to use the smaller set of instructions to be found in a RISC computer.

The aim of computer architects is to design computers which are cheaper and more powerful than their earlier versions. If we review the history of computer families, we find that the most common architectural change is the trend towards even more complex machines. The processors are classified as RISC and CISC based on the design philosophy. They are characterised by the number of instructions supported, design of control unit (hardwired/programmed) and integration of other [186]

Reduced Instruction Set Computer Architecture

resources inside the chip. The RISC processors are considered to support a smaller list of simpler instructions and hardwired control unit which help in fast execution., as it takes shorter duration in finding the requisite instruction.

8.2.1 Importance of RISC Processors

RISC processors exhibit certain distinct features compared to the CISC processors. They include :

- Few Powerful Instructions
- Single clock Cycle execution (for most instruction)
- Register-based execution
- Highly Pipelined Superscalar Architecture
- Fixed Instruction Format and fixed length instruction
- Large register set or register windows
- Hardwired control Unit
- Highly Integrated Architecture

Examples of RISC processor

RISC has fewer design bugs, its simple instructions reduce design time. Thus, because of all the above important reasons RISC processors have become very popular. Some of the RISC processors are :

i860, SPARC Processors

Sun 4/350 SPARCserver 350, Sun 4/360 SPARCserver 360, Sun 4/370 SPARCserver 370, Sun 4/20, SPARCstation SLC, Sun 4/40 SPARCstation IPC, Sun 4/75, SPARCstation 2.

PowerPC Processors (Power refers to Optimisation with enhanced RISC).

MPC755, MPC7400/7410, MPC745X, MPC7450, MPC8240, MPC8245.

Titanium-IA64 Processor

RISC processors are used to build supercomputers for high performance computing. For example, i860 family of processors are used to build PARAM (PARALLEL Machine), PARALLEL Super Computer of Centre for Development of Advanced Computing (C-DAC), India.

8.2.2 Reasons for Increased Complexity

The reasons for increased complexity are :

Speed gap between Memory and CPU

In the past, there was a big gap between the speed of a processor and memory. Thus, a subroutine execution for an instruction, for example floating point addition, may have to follow a lengthy instruction sequence. The question is, if we [187]

Reduced Instruction Set Computer Architecture

make it a machine instruction then only one instruction fetch will be required and rest will be done with control unit sequence. Thus, a "higher level" instruction can be added to machines in an attempt to improve performance.

However, today the Main memory is supported with Cache technology. Cache memories have reduced the difference between the CPU and the memory speed and, therefore, an instruction execution through a subroutine step may not be that difficult. Pipelining can further enhance such speed.

Microcode technology versus VLSI Technology

The control unit of a computer can be constructed using two ways: create micro-program that execute micro-instructions or build circuits (hardwired) for each instruction execution. Micro-programmed control allows the implementation of complex architectures more cost effective than hardwired control as the cost to expand an instruction set is very small, only a few more microinstructions for the control store. Thus it may be reasoned that moving subroutines like string editing, integer to floating point number conversion and mathematical evaluations such as polynomial evaluation to control unit micro-program is more cost effective.

Code Density versus Fast execution of programs

The memory was very large and expensive in the older computer. Thus there was a need of less memory utilization, that is, it was cost effective to have smaller

compact programs. Thus, it was thought that the instruction set should be more complex, so that programs are smaller. However, increased complexity of instruction sets had resulted in instruction sets and addressing modes requiring more bits to represent them. It is stated that the code compaction is important, but the cost of 10 percent more memory is often far less than the cost of reducing code by 10 percent out of the CPU architecture innovations.

The smaller programs are advantageous because they require smaller RAM space. However, today memory is very inexpensive, this potential advantage today is not so compelling. More important, small programs should improve performance because fewer instructions mean fewer instruction bytes to be fetched.

Support for High-Level Language

With the advent and use of more and more higher level languages, manufacturers had provided more powerful instructions to support them. It was argued that a stronger instruction set would reduce the software crisis and would simplify the compilers. Another important reason for such a movement was the desire to improve performance.

However, even though the instructions that were closer to the high level languages were implemented in Complex Instruction Set Computers (CISCs), still it [188]

Reduced Instruction Set Computer Architecture

was hard to exploit these instructions since the compilers were needed to find those conditions that exactly fit those constructs. In addition, the task of optimising the generated code to minimise code size, reduce instruction execution count, and enhance pipelining is much more difficult with such a complex instruction set.

Another motivation for increasingly complex instructions sets was that the complex HLL operation would execute more quickly as a single machine instruction rather than as a series of more primitive instructions. CISC makes the more complex control unit with larger micro program control store to accommodate a richer instruction set. This increases the execution time for simpler instructions. Thus, it is far from clear that the trend to complex instruction sets is appropriate.

8.2.3 High Level Language Program Characteristics

A high-level language system can be implemented mostly by hardware or mostly by software, provided the system hides any lower level details from the programmer. Thus, a cost-effective system can be built by deciding what pieces of the system should be in hardware and what pieces in software. To ascertain the above, it may be a good idea to find program characteristics on general computers. Some of the basic findings about the program characteristics are :

Variables Operations Procedure Calls

Integer Constants 10-25% Simple assignment 35-45% Most time consuming operation

Scalar Variables 50-60% Function call 10-15% Most of the functions

are called with fewer
arguments and have
fewer local variables
Data Structures 15-30% Looping constructs 2-6%
(Array, Stacks, Queues etc)
Conditional statements 35-45%
GOTO FEW Others 1-5%

Figure 1 : Common Program Characteristics

* (The data used above are approximate figures)

Observations

- Integer constants appeared almost as frequently as arrays or structures.
- Most of the scalars were found to be local variables whereas most of the arrays or structures were global variables.
- Most of the dynamically called procedures pass lower than six arguments.
- The numbers of scalar variables are less than six.

[189]

Reduced Instruction Set Computer Architecture

- A good machine design should attempt to optimize the performance of most time consuming features of high-level programs.
- Performance can be improved by more register references rather than having more memory references.
- There should be an optimized instructional pipeline such that any change in flow of execution is taken care of.

8.3 RISC Architecture

Some important considerations of RISC architecture are :

1. The RISC functions are to be kept simple as possible until there is the requirement of complexity. If new operations need to be added then it should be evaluated that if execution time of an instruction increases by 10 per cent then the size of the code should reduce 10 per cent. This is required to balance the simplicity of the code.
2. A simple instruction may be executed at the same speed as that of a micro-instruction because micro-instructions stored in the control unit cannot be faster than simple instructions. This is due to the fact that the cache is built on the same technology as control unit.
3. The runtime library of RISC has all the characteristics of functions in microcode, except that it is easier to change. In general micro codes are difficult to change.
4. Pipelined execution gives a peak performance of one instruction every step. The longest step determines the performance rate of the pipelined machine, so ideally each pipeline step should take the same amount of time.
5. RISC compilers try to remove as much work as possible during compile time so

that simple instructions can be used. For example, RISC compilers try to keep operands in registers so that simple register-to-register instructions can be used. RISC compilers keep operands that will be reused in register, rather than repeating a memory access or a calculation. They, therefore, use LOADs and STOREs to access memory so that operands are not implicitly discarded after being fetched.

Therefore, the RISC architecture were designed with the following features :

One instruction per cycle : A machine cycle is the time taken to fetch two operands from register, perform the ALU operation on them and store the result in a register. Thus, RISC instruction execution takes about the same time as the microinstructions on CISC machines. With such simple instruction execution rather

[190]

Reduced Instruction Set Computer Architecture

than micro-instructions, it can use fast logic circuits for control unit, thus increasing the execution efficiency further.

Register-to-register operands : In RISC machines the operation that access memories are LOAD and STORE. All other operands are kept in registers. This design feature simplifies the instruction set and, therefore, simplifies the control unit. For example, a RISC instruction set may include only one or two ADD instructions (e.g. integer add and add with carry); on the other hand a CISC machine can have 25 add instructions involving different addressing modes. Another benefit is that RISC encourages the optimization of register use, so that frequently used operands remain in registers.

Simple addressing modes : Another characteristic is the use of simple addressing modes. The RISC machines use simple register addressing having displacement and PC relative modes. More complex modes are synthesized in software from these simple ones. Again, this feature also simplifies the instruction set and the control unit.

Simple instruction formats : RISC uses simple instruction formats.

Generally, only one or a few instruction formats are used. In such machines the instruction length is fixed and aligned on word boundaries. In addition, the field locations can also be fixed. Such an instruction format has a number of benefits. With fixed fields, opcode decoding and register operand accessing can occur in parallel. Such a design has many advantages. These are :

- It simplifies the control unit
- Simple fetching as memory words of equal size are to be fetched
- Instructions are not across page boundaries.

Thus, RISC is potentially a very strong architecture. It has high performance potential and can support VLSI implementation. Let us discuss these points in more detail.

Performance using optimizing compilers : As the instructions are simple

the compilers can be developed for efficient code organization also maximizing register utilization etc. Sometimes even the part of the complex instruction can be executed during the compile time.

High performance of Instruction execution : While mapping of HLL to machine instruction the compiler favours relatively simple instructions. In addition, the control unit design is simple and it uses little or no micro-instructions, thus could execute simple instructions faster than a comparable CISC. Simple instructions support better possibilities of using instruction pipelining.

[191]

Reduced Instruction Set Computer Architecture

VLSI Implementation of Control Unit : A major potential benefit of RISC is the VLSI implementation of microprocessor. The VLSI Technology has reduced the delays of transfer of information among CPU components that resulted in a microprocessor. The delays across chips are higher than delay within a chip; thus, it may be a good idea to have the rare functions built on a separate chip. RISC chips are designed with this consideration. In general, a typical microprocessor dedicates about half of its area to the control store in a micro-programmed control unit. The RISC chip devotes only about 6% of its area to the control unit. Another related issue is the time taken to design and implement a processor. A VLSI processor is difficult to develop, as the designer must perform circuit design, layout, and modeling at the device level. With reduced instruction set architecture, this processor is far easier to build.

8.4 The Use of Large Register File

The register storage is the faster storage device, faster than even the main memory and the cache. Thus, a strategy is needed that will allow the most frequently accessed operands to be kept in registers and to minimize register memory operations.

Two basic approaches are possible, one based on the software and the other based on the hardware. The software approach is to rely on the compiler to maximize register usage. The compiler will attempt to allocate the registers to those variables that will be used most in the given time period. This approach requires the use of sophisticated program-analysis algorithms. The hardware approach is simply to use more registers so that more variables can be held in registers for longer periods of time. RISC follows the hardware approach.

Since most operand references are to local scalars, the obvious approach is to store these in registers, with perhaps a few registers reserved for global variables. The problem is the definition of local changes with each procedure call and return, operations that occur frequently. On every call, local variables must be saved from the registers into memory, so that the registers can be reused by the called program. Furthermore, the parameters must be passed. On return, the variables of the parent program must be restored and the results must be passed back to the

parent program.

RISC takes care of these with the help of register windows. Multiple small sets of registers are used, each assigned to different procedure. A procedure call automatically switches the CPU to use a different fixed size window of registers, rather than saving registers in memory. Windows for adjacent procedures are overlapped to allow parameter passing.

[192]

Reduced Instruction Set Computer Architecture

Physical Register Logical Register

PROC A

R31 A

HIGHA

137

132

R26 A

R25 A

LOCALA

131

122 R16 A PROC B

121 LOWA R15 A R31 B

116 HIGHA R10 A R26 B

R25 B

LOCALA

115

104 R16 B PROC C

105 LOWA R15 B R31 C

102 HXHC R10 B R26 C

R25 C

LOCALC

99

10

R16 C

R15 C

LOWC

19

14

R10 C

R9A R9B R9C

LOCAL

9

R0A R0B R0C

0

Figure 2 : Use of three overlapped register windows

Thus the register file, organized in the form as above, is small fast register buffer that holds most of the variables that are likely to be used heavily. From this point of view the register file acts almost like a caches memory.

Characteristics of large-register-file and cache organizations

Large Register File Cache

Hold local variables for almost all Recently used local variables are fetched functions. This saves time. from main memory for any further use.

Dynamic use optimises memory.

The variables are individual. The transfer from memory is block wise.

Global variables are assigned by the It stores recently used variables. It compilers. cannot keep track of future use.

Save/restore needed only after the Save/restore based on cache replacement maximum call nesting is over (that is n-1 algorithms.

open windows).

It follows faster register addressing It is memory addressing.

Table 1 : Characteristics of Large-Register file and cache organisation

[193]

Reduced Instruction Set Computer Architecture

All points above basically show comparative equality. The basic difference is due to addressing overhead of the two approaches.

The following figure shows the difference. Small register (R) address is added with current window Pointer W#. This generates the address in register file, which is decoded by decoder for register access. On the other hand Cache reference will be generated from a long memory address, which first goes through comparison logic to ascertain the presence of data, and if the data is present it goes through the select circuit. Thus, for simple variables access register file is superior to cache memory.

However, even in RISC computer, performance can be enhanced by the addition of instruction cache.

Registers

Instruction R

Data

W# Decoder

(a) Windows based Register file

Instruction Cache

A Tag Data

A >> R

Compare

Select

DATA

(b) Cache Reference

Figure 3 : Referencing a local Simple Variables

[194]

Reduced Instruction Set Computer Architecture

8.5 Comments on RISC (Comparative Analysis of RISC and CISC)

Some of the comments that are asked for RISC architectures are :

CISCs provide better support for high-level languages than RISC as they include high-level language constructs.

Though CISC architecture tries to narrow the gap between assembly and High Level Language (HLL), however, this support is costly. In fact the support can be measured as the inverse of the costs of using typical HLL constructions on a particular machine. If the architect provides a feature that looks like the HLL construct but runs slowly, or has many options, the compiler writer may omit the feature, or even, the HLL programmer may avoid the construct, as it is slow and cumbersome. Thus, the comment above does not hold.

It is more difficult to write a compiler for a RISC than a CISC.

If an instruction can be executed in more ways than one, then more cases must be considered. For it the compiler writer needed to balance the speed of the compilers to get good code. In CISCs, compilers need to analyze the potential usage of all available instruction, which is time consuming. Thus, it is recommended that there is at least one good way to do something. In RISC, there are few choices; for example, if an operand is in memory it must first be loaded into a register. Thus, RISC requires simple case analysis, which means a simple compiler, although more machine instructions will be generated in each case.

RISC is tailored for C language and will not work well with other high level languages.

But the studies of other high level languages found that the most frequently executed operations in other languages are also the same as simple HLL constructs found in C, for which RISC has been optimized Unless a HLL changes the paradigm of programming we will get similar result.

The good performance is due to the overlapped register windows; the reduced instruction set has nothing to do with it.

Certainly, a major portion of the speed is due to the overlapped register windows of the RISC that provide support for function calls. However, please note this register windows is possible due to reduction in control unit size from 50 to 6

[195]

Reduced Instruction Set Computer Architecture

per cent. In addition, the control is simple in RISC than CISC, this further helping the simple instructions to execute faster.

8.6 RISC Pipelining

Pipelining, a standard feature in RISC processors, is much like an assembly line. Because the processor works on different steps of the instruction at the same time more instructions can be executed in a shorter period of time. Different processors have different numbers of steps, they are basically variations of these five, used in the MIPS R3000 processor :

1. fetch instructions from memory
2. read registers and decode the instruction
3. execute the instruction or calculate an address
4. access an operand in data memory
5. write the result into a register

Pipelining is used for enhancing the overall performance. Let us consider Instruction in the context of RISC architecture. In RISC machines most of the operations are register. Therefore, the instructions can be executed in two phases :

F : Instruction Fetch to get the instruction.

E : Instruction Execute on register operands and store the results in register.

In general, the memory access in RISC is performed through LOAD and STORE operations. For such instructions the following steps may be needed :

F : Instruction Fetch to get the instruction

E : Effective address calculation for the desired memory operand

D : Memory to register or register to memory data transfer through bus.

Please note that the pipeline above is not running at its full capacity. This is because of the following problems :

□ We are assuming a single port memory thus only one memory access is allowed at a time. Thus, Fetch and Data transfer operations cannot occur at the same time. Thus, you may notice blank in the time slot 3, 5 etc.

□ The last instruction is an unconditional jump. Please note that after this instruction the next instruction of the calling program will be executed.

Although not visible in this example a branch instruction interrupts the sequential flow of instruction. Thus, causing inefficiencies in the pipelined execution.

[196]

Reduced Instruction Set Computer Architecture

This pipeline can simply be improved by allowing two memory accesses at a time.

Pipeline optimization

In order to make processors even faster, various methods of optimizing pipelines have been devised.

Super-pipelining refer to dividing the pipeline into more steps. The more pipe stages there are, the faster the pipeline is because each stage is then shorter. Ideally, a pipeline with five stages should be five times faster than a non-pipelined processor (or rather, a pipeline with one stage). The instructions are executed at the speed at which each stage is completed, and each stage takes one fifth of the amount of time that the non-pipelined instruction takes. Thus, a processor with an 8-step pipeline (the MIPS R4000) will be even faster than its 5-step counterpart. The MIPS R4000 chops its pipeline into more pieces by dividing some steps into two. Instruction fetching, for example, is now done in two stages rather than one. The states are as shown :

1. Instruction Fetch (First Half)
2. Instruction Fetch (Second Half)
3. Register Fetch
4. Instruction Execute
5. Data Cache Access (First Half)
6. Data Cache Access (Second Half)
7. Tag Check
8. Write Back

Superscalar pipelining involves multiple pipelines in parallel. Internal components of the processor are replicated so it can launch multiple instructions in some or all of its pipeline stages. The RISC System/6000 has a forked pipeline with different paths for floating-point and integer instructions. If there is a mixture of both types in a program, the processor can keep both forks running simultaneously. Both types of instructions share two initial stages (Instruction Fetch and Instruction Dispatch) before they fork. Often, however, superscalar pipelining refers to multiple copies of all pipeline. Many of today's machines attempt to find two to six instructions that it can execute in every pipeline stage. If some of the instructions are dependent, however, only the first instruction or instructions are issued.

[197]

Reduced Instruction Set Computer Architecture

Dynamic pipelines have the capability to schedule around stalls. A dynamic pipeline is divided into three units: the instruction fetch and decode unit, five to ten execute or functional units, and a commit unit. Each execute unit has reservation stations, which act as buffers and hold the operands and operations.

Instruction fetch and

In-order issue

decode unit

Reservation Reservation Reservation Reservation

station station station station

Functional Floating Load/ Out-of-order

units Integer Integer execute

point store

In-order commit

Commit unit

Figure 4 : Dynamic Pipelining

While the functional units have the freedom to execute out of order, the instruction fetch/decode and commit units must operate in-order to maintain simple pipeline behaviour. When the instruction is executed and the result is calculated, the commit unit decides when it is safe to store the result. If a stall occurs, the processor can schedule other instructions to be executed until the stall is resolved. This, coupled with the efficiency of multiple units executing instructions simultaneously, makes a dynamic pipeline an attractive alternative.

RISC machines can employ a very efficient pipeline scheme because of the simple and regular instructions. Like all other instruction pipelines RISC pipeline suffer from the problems of data dependencies and branching instructions. RISC optimizes this problem by using a technique called delayed branching.

One of the common techniques used to avoid branch penalty is to pre-fetch the branch destination also. RISC follows a branch optimization technique called delayed jump as shown in the example given below :

[198]

Reduced Instruction Set Computer Architecture

LOAD R1 □ M (1) F E D

LOAD R2 □□M (2) F E D

SUB RS □ R1 - R2 F E

IF RS < 0 Return F E

ADD RA □□R1 + R2 F E

STOR □ M(A) F E D

RETURN F E

Figure 5 : Delayed Branch

Finally, let us summarize the basic differences between CISC and RISC architecture. The following table lists these differences :

CISC RISC

1 Complex instructions taking multiple Simple instructions taking 1 cycle cycles

2 Any instruction may reference Only LOADS/STORES reference memory memory

3 Not pipelined or less pipelined Highly pipelined

4 Instructions interpreted by the Instructions executed by the

microprogram hardware

5 Variable format instructions Fixed format instructions

6 Many instructions and modes Few instructions and modes

7 Complexity in the microprogram Complexity is in the compiler

8 Single register set Multiple register sets

Table 2 : Difference between RISC and CISC

Source :

www.egr.msu.edu/classes/ece482/Teams/97fall/xdesign2/arm/andy.doc

8.7 Summary

In this unit we have given details of RISC. RISC represents new styles of computers that take less time to build yet provide a higher performance. While traditional machines support High Level Languages (HLLs) with instruction, this [199]

Reduced Instruction Set Computer Architecture

machine supports the use of HLLs with instructions that HLL compilers can use efficiently. The loss of complexity has not reduced RISC's functionality; the chosen subset, especially when combined with the register window scheme, emulates more complex machines. It also appears that we can build such a single chip computer much sooner and with much less effort than traditional architectures.

Thus, we see that because of all the features discussed above, the RISC architecture should prove to be far superior to even the most complex CISC architecture. In this unit we have also covered the details of the pipelined features of the RISC architecture, which further strengthen our arguments for the support of this architecture.

8.8 Questions

1. What is RISC ? Describe the importance of RISC processor.
2. What is the for Increased complexity ? Explain.
3. Explain some important considerations of RISC architecture.
4. What is Large Register File ? Describe the characteristics of Large Register file and cache organisation.
5. Explain RISC Pipelining with an example. What is delayed branching ?

8.9 Suggested Readings

1. Computer Architecture and Organisation by John P. Hayes.

□□□□

[200]

Unit-02/Lecture-01

Control Unit Organization

Control unit

The control unit is a component of a computer's central processing unit (CPU) that directs

operation of the processor. It tells the computer's memory, arithmetic/logic unit and input and output devices how to respond to a program's instructions.

It directs the operation of the other units by providing timing and control signals.[citation needed] All computer resources are managed by the CU (Control Unit).[citation needed] It directs the flow of data between the Central Processing Unit (CPU) and the other devices. John von Neumann included the control unit as part of the von Neumann architecture. In modern computer designs, the control unit is typically an internal part of the CPU with its overall role and operation unchanged.

Functions of the Control Unit

The Control Unit is the circuitry that controls the flow of data through the processor, and coordinates the activities of the other units within it.[citation needed] In a way, it is the "brain within the brain", as it controls what happens inside the processor, which in turn controls the rest of the computer.[citation needed] The examples of devices that require a Control Unit are CPUs and graphics processing units (GPUs).[citation needed] The Control Unit receives external instructions or commands which it converts into a sequence of control signals that the Control Unit applies to the data path to implement a sequence of register-transfer level operations Control Unit.

The Control Unit (CU) is generally a sizable collection of complex digital circuitry interconnecting and controlling the many execution units contained within a CPU.[citation needed] The CU is normally the first CPU unit to accept from an externally stored computer program, a single instruction, based on the CPU's instruction set, then decode this individual instruction into several sequential steps (fetching addresses/data from registers/memory, managing execution [i.e. data sent to the ALU or I/O], and storing the resulting data back into registers/memory) that controls and coordinates the CPU's interworks.[citation needed] These detailed steps from the CU dictate we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

which of the numerous CPU's interconnecting hardware control signals to enable/disable or which CPU units are selected/de-selected and the unit's proper order of execution as required by the instruction's operation.[citation needed] Additionally, the CU's orderly hardware coordination properly sequences these control signals then configures the many hardware units comprising the CPU, directing how data should also be moved, changed, and stored outside the CPU (i.e. memory) according to the instruction's objective.[citation needed] Depending on the type of instruction entering the CU, the order and number of sequential steps produced by the CU could vary the selection and configuration of which parts of the CPU's hardware are utilized to achieve the instruction's objective (mainly moving, storing, and modifying data within the CPU).[citation needed] This one feature, that efficiently uses just software instructions to control/select/configure a computer's CPU hardware (via the CU) and eventually manipulates a program's data, is a significant reason most modern computers are flexible and universal when running various programs. As compared to some 1930s or 1940s computers without a proper CU, they often required rewiring their hardware when changing programs.[citation needed] This CU instruction decode process is then repeated when the Program Counter is incremented to the

next stored program address and the new instruction enters the CU from that address, and so on till the programs end.

Other more advanced forms of Control Units manage the translation of instructions (but not the data containing portion) into several micro-instructions and the CU manages the scheduling of the micro-instructions between the selected execution units to which the data is then channeled and changed according to the execution unit's function (i.e., ALU contains several functions).[citation needed] On some processors, the Control Unit may be further broken down into additional units, such as an instruction unit or scheduling unit to handle scheduling, or a retirement unit to deal with results coming from the instruction pipeline.[citation needed] Again, the Control Unit orchestrates the main functions of the CPU: carrying out stored instructions in the software program then directing the flow of data throughout the computer based upon these instructions (roughly likened to how traffic lights will systematically control the flow of cars [containing data] to different locations within the traffic grid [CPU] until it parks at the desired parking spot [memory address/register].[citation needed] The car occupants [data] then go into the building [execution unit] and comes back changed in some way then get back into the car and returns to another location via the controlled traffic grid).

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Fig 2.1 Combinational Logic Circuits

Hardwired control unit

Hardwired control units are implemented through use of sequential logic units, featuring a finite number of gates that can generate specific results based on the instructions that were used to invoke those responses. Hardwired control units are generally faster than micro programmed designs. Their design uses a fixed architecture—it requires changes in the wiring if the instruction set is modified or changed.[citation needed] This architecture is preferred in reduced instruction set computers (RISC) as they use a simpler instruction set.

A controller that uses this approach can operate at high speed; however, it has little flexibility, and the complexity of the instruction set it can implement is limited.

The hardwired approach has become less popular as computers have evolved. Previously, control units for CPUs used ad-hoc logic, and they were difficult to design.

Micro program control unit:

The idea of microprogramming was introduced by Maurice Wilkes in 1951 as an intermediate level to execute computer program instructions. Micro programs were organized as a sequence of microinstructions and stored in special control memory. The algorithm for the micro program

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

control unit is usually specified by flowchart description.[4] The main advantage of the micro program control unit is the simplicity of its structure. Outputs of the controller are organized in microinstructions and they can be easily replaced.

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Fig RGPV QUESTIONS Year Marks

2.2S.NO

Q.1 Discuss in brief microprogram control unit and June 2012 7

hardwired control unit?

Q.2 What is need of control unit in a computer? Dec 2010 4

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Unit-02/Lecture-02

Control Memory

- The control unit in a digital computer initiates sequences of micro operations.
- The complexity of the digital system is derived form the number of sequences that are performed
- When the control signals are generated by hardware, it is hardwired
- In a bus-oriented system, the control signals that specify micro operations are groups of bits that select the paths in multiplexers, decoders, and ALUs.
- The control unit initiates a series of sequential steps of micro operations
- The control variables can be represented by a string of 1's and 0's called a control word
- A micro programmed control unit is a control unit whose binary control variables are stored in memory .
- The control unit initiates a series of sequential steps of micro operations
- The control variables can be represented by a string of 1's and 0's called a
- control word
- A micro programmed control unit is a control unit whose binary control variables are stored in memory
- A sequence of microinstructions constitutes a micro program
- The control memory can be a read-only memory
- Dynamic microprogramming permits a micro program to be loaded and uses a writable control memory
- A computer with a micro programmed control unit will have two separate
- memories: a main memory and a control memory
- The micro program consists of microinstructions that specify various internal control signals for execution of register micro operations
- These microinstructions generate the micro operations to:
- fetch the instruction from main memory
- evaluate the effective address
- execute the operation
- Return control to the fetch phase for the next instruction.
- The control memory address register specifies the address of the microinstruction
- The control data register holds the microinstruction read from memory
- The microinstruction contains a control word that specifies one or more
- Micro operations for the data processor

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Fig 2.3 Control unit

Address Sequencing

Microinstructions are usually stored in groups where each group specifies a routine, where each routine specifies how to carry out an instruction. Each routine must be able to branch to the next routine in the sequence. An initial address is loaded into the CAR when power is turned on; this is usually the address of the first microinstruction in the instruction fetch routine. Next, the control unit must determine the effective address of the instruction.

When instruction execution is finished, control must be return to the fetch routine. This is done using an unconditional branch. Addressing sequencing capabilities of control memory include Incrementing the CAR Unconditional and conditional branching (depending on status bit). Mapping instruction bits into control memory addresses Handling subroutine calls and returns.

Mapping

The next step is to generate the micro operations that executed the instruction. This involves we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

taking the instructions op code and transforming it into an address for the instructions micro program in control memory. This process is called mapping.

While microinstruction sequences are usually determined by incrementing the CAR, this is not always the case. If the processor's control unit can support subroutines in a micro program, it will need an external register for storing return addresses

Fig 2.4 Selection of Address For control memory

S.NO RGPV QUESTIONS Year Marks

Q.1 Explain the concept of address sequencing .Also explain June 2012 7 mapping of an instruction.

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Unit-02/Lecture-03

Microinstruction Formats:

Two basic formats

1. Horizontal Microinstruction Format

One bit for each possible signal that might need to be generated by any microinstruction -leads to the

fastest execution:

Requires d bits if there are d possible destinations plus s bits if there are s possible sources.

2. Vertical Microinstruction Format

Mutually exclusive operations grouped together and encoded in binary. Reduces number of bits in microinstruction. Each vertically encoded field needs a decoder

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Example

Suppose there were up to 15 possible sources and destinations (PC, MAR, MDR, IR). Four bits needed to

specify which one:

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

S.NO RGPV QUESTIONS Year Marks

Q.1 Define the following: June 2012 7

(i) Micro -operation

(ii) Microcode

(iii) Microinstruction

(iv) MicroProgram

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Unit-02/Lecture-04

Micro Program sequencer

In computer architecture and engineering, a sequencer or micro sequencer generates the addresses used to

step through the micro program of a control store. It is used as a part of the control unit of a CPU or as a

stand-alone generator for address ranges.

Usually the addresses are generated by some combination of a counter, a field from a microinstruction, and

some subset of the instruction register. A counter is used for the typical case, that the next microinstruction

is the one to execute. A field from the microinstruction is used for jumps, or other logic.

Since CPUs implement an instruction set, it's very useful to be able to decode the instruction's bits directly

into the sequencer, to select a set of microinstructions to perform a CPU's instructions.

Most modern CPUs are considerably more complex than this description suggests. They tend to have

multiple cooperating micro machines with specialized logic to detect and handle interference between the

micro machines.

Or

A micro program sequencer for micro programmed control unit develops micro program consecutive

addresses, branches to subroutines with address saving and possible return to micro program, as well as

interrupting micro program forcings with address saving of the interrupted micro programs.

In order to allow the double saving of micro program and subroutine addresses in case of concurrent

interruptions and branches, the sequencer is provided with two address generation loops each including a

register. The two loops have a common portion to which they accede through a multiplexer (23).

The first loop (23, 25, 22, 21, 30, 31) is further coupled to a saving register stack (20). While the first loop executes the saving of a micro program address and the latching of a branch address received from the second loop, the second loop (23, 25, 24, 39, 17, 18, 42, 19, 27, 29) executes a first updating and-, related latching of interrupting micro program address. During the following cycle, by command of the first microinstruction of the interrupting micro program, the second loop performs a first updating and related latch of the interrupting micro program address and the first loop saves into the register stack (20) the branch address and performs a second updating and related latching of the interrupting micro program address.

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

MultiProgramming

Microcode is a layer of hardware-level instructions that implement higher-level machine code instructions or internal state machine sequencing in many digital processing elements. Microcode is used in general central processing units, in more specialized processors such as microcontrollers, digital signal processors, channel controllers, disk controllers, network interface controllers, network processors, graphics processing units, and in other hardware.

Microcode typically resides in special high-speed memory and translates machine instructions, state machine data or other input into sequences of detailed circuit-level operations. It separates the machine instructions from the underlying electronics so that instructions can be designed and altered more freely. It also facilitates the building of complex multi-step instructions, while reducing the complexity of computer circuits. Writing microcode is often called microprogramming and the microcode in a particular processor implementation is sometimes called a microprogram.

Engineers normally write the microcode during the design phase of a processor, storing it in a ROM (read-only memory) or PLA (programmable logic array)[1] structure, or in a combination of both. However, machines also exist that have some (or all) microcode stored in SRAM or flash memory. This is traditionally denoted a "writeable control store" in the context of computers. Complex digital processors may also

employ more than one (possibly microcode-based) control unit in order to delegate sub-tasks that must be performed (more or less) asynchronously in parallel. A high-level programmer, or even an assembly programmer, does not normally see or change microcode. Unlike machine code, which often retains some compatibility among different processors in a family, microcode only runs on the exact electronic circuitry for which it is designed, as it constitutes an inherent part of the particular processor design itself. More extensive microcoding allows small and simple microarchitectures to emulate more powerful architectures with wider word length, more execution units and so on – a relatively simple way to achieve software compatibility between different products in a processor family.

Some hardware vendors, especially IBM, use the term "microcode" as a synonym for "firmware". That way, all code in a device is termed "microcode" regardless of it being microcode or machine code; for example, hard disk drives are said to have their microcode updated, though they typically contain both microcode and firmware.

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

S.NO RGPV QUESTIONS Year Marks

Q.1 With a neat block diagram explain the working principle Dec 2010 5
of micro program sequencer

Q.2 Draw the format of a microinstruction and explain how a June 2010 7
microprogram sequencer works.

Q.3 Write a brief note on microprogram sequencer June 2014 2

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Unit-02/Lecture-05

Arithmetic Logic Unit

Arithmetic Logic Unit, ALU is one of the many components within a computer processor. The ALU performs mathematical, logical, and decision operations in a computer and is the final processing performed by the processor. After the information has been processed by the ALU, it is sent to the computer memory. In some computer processors, the ALU is divided into an AU and LU. The AU performs the arithmetic operations and the LU performs the logical operations.

Fig 2.5 Arithmetic Logic Unit

An arithmetic logic unit (ALU) is a digital circuit that performs integer arithmetic and logical operations. The

ALU is a fundamental building block of the central processing unit of a computer, and even the simplest

microprocessors contain one for purposes such as maintaining timers. The processors found inside modern

CPUs and graphics processing units (GPUs) accommodate very powerful and very complex ALUs; a single

component may contain a number of ALUs.

An ALU must process numbers using the same formats as the rest of the digital circuit. The format of modern processors is almost always the two's complement binary number representation.

Early computers used a wide variety of number systems, including ones' complement, two's complement, sign-magnitude format, and even true decimal systems, with various representation of the digits.

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

The ones' complement and two's complement number systems allow for subtraction to be accomplished by adding the negative of a number in a very simple way which negates the need for specialized circuits to do subtraction; however, calculating the negative in two's complement requires adding a one to the low order bit and propagating the carry. An alternative way to do the subtraction of $A - B$ is to present a one to the right input of the adder and use $\neg B$ rather than B as the second input. The arithmetic, logic and shift circuits introduced in previous sections can be combined into one ALU with common selection

An arithmetic logic unit (ALU) is a digital circuit used to perform arithmetic and logic operations. It represents the fundamental building block of the central processing unit (CPU) of a computer.

Modern CPUs contain very powerful and complex ALUs. In addition to ALUs, modern CPUs contain a control unit (CU). Most of the operations of a CPU are performed by one or more ALUs, which load data from input registers. A register is a small amount of storage available as part of a CPU. The control unit tells the ALU what operation to perform on that data and the ALU stores the result in an output register. The control unit moves the data between these registers, the ALU and memory.

ALU working

An ALU performs basic arithmetic and logic operations. Examples of arithmetic operations are addition, subtraction, multiplication, and division. Examples of logic operations are comparisons of values such as NOT, AND, and OR.

All information in a computer is stored and manipulated in the form of binary numbers, i.e. 0 and 1. Transistor switches are used to manipulate binary numbers, since there are only two possible states of a switch: open or closed. An open transistor, through which there is no current, represents a 0. A closed transistor, through which there is a current, represents a 1. Operations can be accomplished by connecting multiple transistors. One transistor can be used to control a second one, in effect turning the transistor switch on or off depending on the state of the second transistor. This is referred to as a gate, because the arrangement can be used to allow or stop a current. The simplest type of operation is a NOT gate. This uses only a single transistor. It uses a single input and produces a single output, which is always the opposite of the input. The figure below shows the logic of the NOT gate.

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Unit-03/Lecture-06

These computational units process the binary data directly and provide support for arithmetic computations of varying precision. The ALU performs a standard set of arithmetic and logic operations as

well as division. The MAC performs multiply, multiply/add, and multiply/subtract operations. The shifter

performs logical and arithmetic shifts, normalization, denormalization, and other operations.

Addition and Subtraction

Addition is the most common arithmetic operation a processor performs. When two n-bit numbers are

added together, it is always possible to produce a result with $n + 1$ nonzero digits due to a carry from the

leftmost digit. For two's complement addition of two numbers, there are three cases to consider:

□ If both numbers are positive and the result of their addition has a sign bit of 1, then overflow has occurred; otherwise the result is correct.

□ If both numbers are negative and the sign of the result is 0, then overflow has occurred; otherwise

the result is correct.

□ If the numbers are of unlike sign, overflow cannot occur and the result is always correct.

□ For addition use normal binary addition

□ $0+0=\text{sum } 0 \text{ carry } 0$

□ $0+1=\text{sum } 1 \text{ carry } 0$

□ $1+1=\text{sum } 0 \text{ carry } 1$

Monitor MSB for overflow

Overflow cannot occur when adding 2 operands with the different signs. If 2 operand have same sign and

result has a different sign, overflow has occurred. Subtraction: Take 2's complement of subtrahend and

add to minuend i.e. $a - b = a + (b)$, So we only need addition and complement circuits

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Fig. 2.6 Hardware for Addition & Subtraction

S.NO RGPV QUESTIONS Year Marks

Q.1 Draw and explain flowchart for addition and subtraction of June 2009, 7
floating point number June 2012

Q.2 Write down the algorithm for addition and subtraction with June 2011 10
signed-magnitude data. Also draw the flowchart.

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Unit-02/Lecture-07

Multiplication

A complex operation compared with addition and subtraction. Many algorithms are used, esp. for large

numbers. Simple algorithm is the same long multiplication taught in grade school.

Compute partial product for each digit. Add partial products.

Multiplication Example

- Multiplier
- Multiplier
- Partial products
- Note: if multiplier it is odd
- Multiplier value
- otherwise zero
- Product
- Note: feed double length result

Fig 2.7

Multiplication Algorithm

Repeat n times:

If $Q_0 = 1$ Add M into A, store carry in CF

we don't take any liability for the notes correctness. <http://www.rgpvonline.com>

Shift CF, A, Q right one bit so that:

$A_n \leftarrow CF, Q_{n-1} \leftarrow A_0, Q_0$ is lost

Note that during execution Q contains bits from both product and multiplier

Fig 2.8 Flowchart for Unsigned Binary Multiplication

Division

- More popular than multiplication for computers as well as humans. Some processors designed for embedded applications or digital signal processing lack a divide instruction.
- Basic idea of add and shift: shift and subtract.

Unsigned Division algorithm

- Using same registers A, M, Q, output as multiplication
- Results of division are quotient and remainder

Q will hold the quotient

A will hold the remainder

- Initial values

$Q \leftarrow 0$

$A \leftarrow \text{Dividend}$

$M \leftarrow \text{Divisor}$

Count $\leftarrow n$

we don't take any liability for the notes correctness. <http://www.rgpvonline.com>

Fig 2.9 Flowchart for Division

we don't take any liability for the notes correctness. <http://www.rgpvonline.com>

Unit-02/Lecture-08

Floating Point Arithmetic

Arithmetic operations on floating point numbers consist of addition, subtraction, multiplication and division

The operations are done with algorithms similar to those used on sign magnitude integers (because of the

similarity of representation) -- example, only add numbers of the same sign. If the numbers are of opposite sign, must do subtraction.

Arithmetic unit

The arithmetic unit, also called the arithmetic logic unit (ALU), is a component of the central processing unit

(CPU). It is often referred to as the engine of the CPU because it allows the computer to perform mathematical calculations, such as addition, subtraction, and multiplication. The ALU also performs logic

operations, like AND, OR, and NOT. The arithmetic unit works along with the register array, which holds data, when processing any of these operations. The arithmetic unit is comprised of many interconnected elements that are designed to perform specific tasks.

Some central processing units are comprised of two components, an arithmetic unit and a logic unit. Other

processors may have an arithmetic unit for calculating fixed-point operations and another AU for calculating

floating-point computations. Some PCs have a separate chip known as the numeric coprocessor. This

coprocessor contains a floating-point unit for processing floating-point operands. The coprocessor increases

the operating speed of the computer because of the coprocessor ability to perform computation faster and

more efficiently.

RGPV QUESTIONS Year Marks

Q.1 Take an example and explain the design of arithmetic and June 2014 7

logic unit

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

Unit-02/Lecture-09

An Arithmetic Unit is a combinational circuit that performs arithmetic microoperations on a pair of n-bit operands (ex. A[3:0] and B[3:0]). The operations performed by an AU are controlled by a set of function-select inputs. In this design a 4-bit AU with 2 function-select inputs: Select S1 and S0 inputs. The functions performed by the AU are specified in Table A block diagram is given in Figure Table No.2.1

Fig 2. 10 Block Diagram of Arithmetic Unit

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

We will be using two's complement system of notation while dealing with arithmetic operations in our AU. This has a number of advantages over the sign and magnitude representation such as easy addition or subtraction of mixed positive and negative numbers. Recall that the two's complement of a n-bit number N is defined as:

$$2^n - N = (2^n - 1 - N) + 1$$

The last representation gives us an easy way to find two's complement: take the bit wise complement of the number and add 1 to it. As an example, to represent the number -5, we take two's complement of 5 (=0101) as follows,

5 0 1 0 1 --> 1 0 1 0 (bit wise complement)

+1

1 0 1 1 (two's complement)

Numbers represented in two's complement lie within the range - (2^{n-1}) to + $(2^{n-1} - 1)$. For a 4-bit number this means that the number is in the range of -8 to +7. There is a potential problem we still need to be aware of when working with two's complement, namely overflow and underflow as is illustrated in the examples below,

0 1 0 0 (=carry Ci)

+5 0 1 0 1

+4 + 0 1 0 0

+9 0 1 0 0 1 = -7!

Also,

1 0 0 0 (=carry Ci)

-7 1 0 0 1

we don't take any liability for the notes correctness. <http://www.rgpvonline.com>

-2 + 1 1 1 0

-9 1 0 1 1 1 = +7!

Both calculations give the wrong results (-7 instead of +9 or +7 instead of -9) which is caused by the

fact that the result +9 or -9 is out of the allowable range for a 4-bit two's complement number.

Whenever the result is larger than +7 or smaller than -8 there is an overflow or underflow and the result of the addition or subtraction is wrong. Overflow and underflow can be easily detected when the carry out of the most significant stage (i.e. C4) is different from the carry out of the previous stage (i.e. C3).

The inputs A and B have to be presented in two's complement to the inputs of the AU.

RGPV QUESTIONS Year Marks

Q.1 Take an example and explain the design of arithmetic and June 2014 7

logic unit

we dont take any liability for the notes correctness. <http://www.rgpvonline.com>

EE442 Operating Systems Ch. 3 Memory Management

Chapter 3

Memory Management

In a multiprogramming system, in order to share the processor, a number of processes must be kept in memory. Memory management is achieved through memory management algorithms. Each memory management algorithm requires its own hardware support. In this chapter, we shall see the partitioning, paging and segmentation methods.

In order to be able to load programs at anywhere in memory, the compiler must generate relocatable object code. Also we must make it sure that a program in memory, addresses only its own area, and no other program's area. Therefore, some protection mechanism is also needed.

3.1 Fixed Partitioning

Memory In this method, memory is divided into partitions whose sizes are fixed. OS is placed into the OS lowest bytes of memory. Processes are classified on entry to the system according to (n KB) Small their memory they requirements. We need one (3n KB) Process Queue (PQ) for each class of process.

Medium

If a process is selected to allocate memory, then it goes into memory and competes for the processor. The number of fixed partition gives (6n KB) the degree of multiprogramming. Since each

Large

queue has its own memory region, there is no competition between queues for the memory.

Fixed Partitioning with Swapping

This is a version of fixed partitioning that uses RRS with some time quantum. When time quantum for a process expires, it is swapped out of memory to disk and the next process in the corresponding process queue is swapped into the memory.

OS

2K P1 P3

P2

6K

P4 P5

empty

12K empty

Lecture Notes by Uğur Halıcı 18

EE442 Operating Systems Ch. 3 Memory Management

Normally, a process swapped out will eventually be swapped back into the same partition. But this restriction can be relaxed with dynamic relocation.

In some cases, a process executing may request more memory than its partition size. Say we have a 6 KB process running in 6 KB partition and it now requires a more memory of 1 KB. Then, the following policies are possible:

- Return control to the user program. Let the program decide either quit or modify its operation so that it can run (possibly slow) in less space.
- Abort the process. (The user states the maximum amount of memory that the process will need, so it is the user's responsibility to stick to that limit)
- If dynamic relocation is being used, swap the process out to the next largest PQ and locate into that partition when its turn comes.

The main problem with the fixed partitioning method is how to determine the number of partitions, and how to determine their sizes.

If a whole partition is currently not being used, then it is called an external fragmentation. And if a partition is being used by a process requiring some memory smaller than the partition size, then it is called an internal fragmentation.

OS

In this composition of memory, if a

2K P1 (2K)

new process, P3, requiring 8 KB of

6K Empty (6k) External memory comes, although there is fragmentation

enough total space in memory, it can

not be loaded because fragmentation.

12K P2 (9K)

Internal

Empty (3K) fragmentation

3.2 Variable Partitioning

With fixed partitions we have to deal with the problem of determining the number and sizes of partitions to minimize internal and external fragmentation. If we use variable partitioning instead, then partition sizes may vary dynamically.

In the variable partitioning method, we keep a table (linked list) indicating used/free areas in memory. Initially, the whole memory is free and it is considered as one large block. When a new process arrives, the OS searches for a block of free memory large enough for that process. We keep the rest available (free) for the future processes. If a block becomes free, then the OS tries to merge it with its neighbors if they are also free.

There are three algorithms for searching the list of free blocks for a specific amount of

memory.

First Fit : Allocate the first free block that is large enough for the new process. This is a fast algorithm.

Lecture Notes by Uğur Halıcı 19

EE442 Operating Systems Ch. 3 Memory Management

Best Fit : Allocate the smallest block among those that are large enough for the new process. In this method, the OS has to search the entire list, or it can keep it sorted and stop when it hits an entry which has a size larger than the size of new process. This algorithm produces the smallest left over block. However, it requires more time for searching all the list or sorting it.

Worst Fit : Allocate the largest block among those that are large enough for the new process. Again a search of the entire list or sorting it is needed. This algorithm produces the largest over block.

Example 3.1

Consider the following memory map and assume a new process P4 comes with a memory requirement of 3 KB. Locate this process.

OS

P1

<free> 10 KB

a. First fit algorithm allocates from the 10 KB block.

P2 b. Best fit algorithm allocates from the 4 KB block.

<free> 16 KB c. Worst fit algorithm allocates from the 16 KB block.

P3

<free> 4 KB

New memory arrangements with respect to each algorithms will be as follows:

OS OS OS

P1 P1 P1

P4 <free> 10 KB <free> 10 KB

<free> 7 KB P2 P2

P2 <free> 16 KB P4

<free> 16 KB P3 <free> 13 KB

P3 P4 P3

<free> 4 KB <free> 1 KB <free> 4 KB

First Fit Best Fit Worst Fit

At this point, if a new process, P5 of 14K arrives, then it would wait if we used worst fit algorithm, whereas it would be located in cases of the others.

Compaction: Compaction is a method to overcome the external fragmentation problem. All free blocks are brought together as one large block of free space. Compaction requires dynamic relocation. Certainly, compaction has a cost and selection of an optimal compaction

strategy is difficult. One method for compaction is swapping out those processes that are to be moved within the memory, and swapping them into different memory locations.

OS

P1 OS

<free> 20 KB P1

P2 Compaction P2

<free> 7 KB P3

P3 <free> 37 KB

<free> 10 KB

Lecture Notes by Uğur Halıcı 20

EE442 Operating Systems Ch. 3 Memory Management

3.3 Paging

Paging permits a program to allocate noncontiguous blocks of memory. The OS divide programs into pages which are blocks of small and fixed size. Then, it divides the physical memory into frames which are blocks of size equal to page size. The OS uses a page table to map program pages to memory frames. Page size (S) is defined by the hardware. Generally page size is chosen as a power of 2 such as 512 words/page or 4096 words/page etc.

With this arrangement, the words in the program have an address called as logical address. Every logical address is formed of

- A page number p where $p = \text{logical address} \div S$
- An offset d where $d = \text{logical address} \bmod S$

When a logical address $\langle p, d \rangle$ is generated by the processor, first the frame number f corresponding to page p is determined by using the page table and then the physical address is calculated as $(f \cdot S + d)$ and the memory is accessed.

Logical memory Physical memory

P0 page frame Attributes f_0

0 4

P1 1 3 P2 f_1

2 1

P2 3 5 f_2

P3 Page Table P1 f_3

P0 f_4

P3 f_5

The address translation in paging is shown below

Logical address Physical address

$p \ d \ f \ d$

d page frame Attributes d

p f

p f

Logical Memory Page Table Physical Memory

Lecture Notes by Uğur Halıcı 21

EE442 Operating Systems Ch. 3 Memory Management

Example 3.2

Consider the following information to form a physical memory map.

Page Size = 8 words

Physical Memory Size = 128 words

A program of 3 pages where P0 $\in$ f3; P1 $\in$ f6; P2 $\in$ f4

Logical Program Physical Memory

Word 0

Word 1 Page 0 Word 0

... (P0) Page Table Word 1 Frame 3

Word 7 ... (f3)

Word 8 Page Frame Word 7

Word 9 Page 1 0 3 Word 16

... (P1) 1 6 Word 17 Frame 4

Word 15 2 4 ... (f4)

Word 16 Word 23

Word 17 Page 2

... (P2)

Word 23

Word 8

Word 9 Frame 6

... (f6)

Word 15

... ..

Program Logical Offset Page Frame Physical

Line Address Number Number Address

Word 0 00 000 000 00 011 011 000

Word 1 00 001 001 00 011 011 001

... ..

Word 7 00 111 111 00 011 011 111

Word 8 01 000 000 01 110 110 000

Word 9 01 001 001 01 110 110 001

... ..

Word 15 01 111 111 01 110 110 111

Word 16 10 000 000 10 100 100 000

Word 17 10 001 001 10 100 100 001

Word 23 10 111 111 10 100 100 111

How to Implement The Page Table?

Every access to memory should go through the page table. Therefore, it must be implemented in an efficient way.

a. Using fast dedicated registers

Keep page table in fast dedicated registers. Only the OS is able to modify these registers. However, if the page table is large, this method becomes very expensive since requires too many registers.

Lecture Notes by Uğur Halıcı 22

EE442 Operating Systems Ch. 3 Memory Management

Logical address

PTLR: Page Table Length Register

p d

Physical address

f d

YES Access PT Access

p < PTLR? in Registers memory

rat mat

NO

ERROR

Given a logical address to access the word in physical memory, first access the PT stored in registers, which requires register access time (rat), and then find out the physical address and access the physical memory, which requires memory access time (mat). Therefore effective memory access time (emat) becomes:

$emat = rat + mat$

b. Keep the page table in main memory

In this method, the OS keeps a page table in the memory. But this is a time consuming method. Because for every logical memory reference, two memory accesses are required:

1. To access the page table in the memory, in order to find the corresponding frame number.
2. To access the memory word in that frame

Logical address

PTBR: Page Table Base Register

p d PTLR: Page Table Length Register

Physical address

f d

Access PT Access

p < PTLR? entry memory

in Memory

at address mat

PTBR + p

NO

mat

. ERROR

In this approach emat is:

emat = 2 * mat

Lecture Notes by Uğur Halıcı 23

EE442 Operating Systems Ch. 3 Memory Management

c. Use content-addressable associative registers

These are small, high speed registers built in a special way so that they permit an associative search over their contents. That is, all registers may be searched in one machine cycle simultaneously. However, associative registers are quite expensive. So, a small number of them should be used.

Logical address

PTBR: Page Table Base Register

p d PTLR: Page Table Length Register

Physical address

f d

YES Search PT

p < PTLR? in AR FOUND?

YES (hit)

rat

NO

NO (miss) Physical address

f d

ERRO Access PT Access

entry memory

in Memory

at address mat

PTBR + p

mat

When a logical memory reference is made, first the corresponding page number is searched in associative registers. If that page number is found in one associative register (hit) then the corresponding frame number is get, else (miss) the page table in memory is accessed to find the frame number and that <page number, frame number> pair is stored into associative registers. Once the frame number is obtained, the memory word is accessed.

The hit ratio is defined as the percentage of times that a page number is found in associative registers. Hit ratio is important in performance of the system since it affects the effective

memory access time. In the case of finding the page number in associative registers, only one memory access time is required whereas if it cannot be found two memory accesses are needed. So, greater the hit ratio, smaller the effective memory access time. Effective memory access time is calculated as follows:

$$\text{emat} = h * \text{ematHIT} + (1-h) * \text{ematMISS}$$

where

h = The hit ratio

ematHIT = effective memory access time when there is a hit = $\text{rat} + \text{mat}$

ematMISS = effective memory access time when there is a miss = $\text{rat} + \text{mat} + \text{mat}$

Lecture Notes by Uğur Halıcı 24

EE442 Operating Systems Ch. 3 Memory Management

Example 3.3

Assume we have a paging system which uses associative registers. These associative registers have an access time of 30 ns, and the memory access time is 470 ns. The system has a hit ratio of 90 %.

Now, if the page number is found in one of the associative registers, then the effective access time:

$$\text{ematHIT} = 30 + 470 = 500 \text{ ns.}$$

Because one access to associative registers and one access to the main memory is sufficient.

On the other hand, if the page number is not found in associative registers, then the effective access time:

$$\text{ematMISS} = 30 + (2 * 470) = 970 \text{ ns.}$$

Since one access to associative registers and two accesses to the main memory are required.

Then, the emat is calculated as follows:

$$\begin{aligned} \text{emat} &= 0.9 * 500 + 0.1 * 970 \\ &= 450 + 97 = 547 \text{ ns} \end{aligned}$$

Sharing Pages

Sharing pages is possible in a paging system, and is an important advantage of paging. It is possible to share system procedures or programs, user procedures or programs, and possibly data area. Sharing pages is especially advantageous in time-sharing systems. A reentrant program (non-self-modifying code = read only) never changes during execution. So, more than one process can execute the same code at the same time. Each process will have its own data storage and its own copy of registers to hold the data for its own execution of the shared program.

Example 3.4

Consider a system having page size=30 MB. There are 3 users executing an editor program

which is 90 MB (3 pages) in size, with a 30 MB (1 page) data space.

To support these 3 users, the OS must allocate $3 * (90+30) = 360$ MB space. However, if the editor program is reentrant (non-self-modifying code = read only), then it can be shared among the users, and only one copy of the editor program is sufficient. Therefore, only $90 + 30 * 3 = 180$ MB of memory space is enough for this case.

Lecture Notes by Uğur Halıcı 25

EE442 Operating Systems Ch. 3 Memory Management

User-1 PT-1 Physical

P0 e1 Page# Frame# Memory

P1 e2 0 8 f0

P2 e3 1 4 f1

P3 data1 2 5 f2

3 7 f3

f4 e2

User-2 PT-2 f5 e3

P0 e1 Page# Frame# f6

P1 e2 0 8 f7 data1

P2 e3 1 4 f8 e1

P3 data2 2 5 f9

3 12 f10 data3

f11

User-3 PT-3 f12 data 2

P0 e1 Page# Frame# f13

P1 e2 0 8 f14

P2 e3 1 4 f15

P3 data3 2 5

3 10

3.2 Segmentation

In segmentation, programs are divided into variable size segments, instead of fixed size pages. Every logical address is formed of a segment name and an offset within that segment. In practice, segments are numbered. Programs are segmented automatically by the compiler or assembler.

For example, a C compiler will create separate segments for:

1. the code of each function
2. the local variables for each function
3. the global variables.

Main

Func 1 Func 2

Data 1

Data 1

Data 3

Lecture Notes by Uğur Halıcı 26

EE442 Operating Systems Ch. 3 Memory Management

For logical to physical address mapping, a segment table is used. When a logical address $\langle s, d \rangle$ is generated by the processor:

1. Base and limit values corresponding to segment s are determined using the segment table
2. The OS checks whether d is in the limit. ($0 \leq d < \text{limit}$)
3. If so, then the physical address is calculated as $(\text{base} + d)$, and the memory is accessed.

logical address

$s \ d$

base

d

access the Segment S

YES word at

$0 \leq d < \text{limit}$ physical

Seg. # Limit base Attr. address =

NO base + d

ERROR

Example 3.5

Generate the memory map according to the given segment table. Assume the generated logical address is $\langle 1, 123 \rangle$; find the corresponding physical address.

Segment Limit Base physical

memory

0 1500 1000

0

1 200 5500

2 700 6000 1000

3 2000 3500 s_0

2500

3500

Now, check segment table entry for segment 1. s_3

The limit for segment 1 is 200. Since $123 < 200$,

we carry on. The physical address is calculated as $s_1 \ 5500$

$5500 + 123 = 5623$, and the memory word 5623 is 5700

accessed. $s_2 \ 6000$

6700

Segment tables are also implemented in the main memory or in associative registers, in the

same way it is done for page tables.

Lecture Notes by Uğur Halıcı 27

EE442 Operating Systems Ch. 3 Memory Management

Sharing Segments

Also sharing of segments is applicable as in paging. Shared segments should be read only and should be assigned the same segment number.

Example 3.6:

Consider a system in which 3 users executing an editor program which is 1500 KB in size, each having their own data space.

user-1 ST-1

seg lim base

editor 0 1500 1000

1 2000 3500 physical

data-1 memory

0

1000

editor

user-2 2500

ST-2

3500

seg lim base

editor data-1

0 1500 100

1 200 5500

data-2 5500

data-2 5700

data-3 6000

6700

user-3 ST-3

seg lim base

editor 0 1500 100

1 700 6000

data-3

Lecture Notes by Uğur Halıcı 28

EE442 Operating Systems Ch. 3 Memory Management

3.3. Paged segmentation

The idea is to page the segments and eliminate the external fragmentation problem. In paged segmentation the logical address is made of $\langle s, p, d \rangle$ triplet. The ST entry for segment S now, contains:

- the length of segment S
- the base address of the PT for segment S.

There is a separate PT for every segment. On the average, now there is half a page of internal fragmentation per segment. However, more table space is needed. In the worst case, again three memory accesses are needed for each memory reference.

The flowchart for accessing a word with logical address $\langle s, p, d \rangle$ is shown below.

$0 \leq pd \leq \text{limit PT for}$

$s \quad pd \quad p \quad d$ segment S

ST

STBR NO

ERROR f

limit base

f d

d

f

STBR: Segment Table Base Register

QUESTIONS

1. Why do we need memory management and CPU scheduling algorithms for a multiuser system ? Can we do without these algorithms? Explain briefly.
2. a. Explain the terms: internal fragmentation and external fragmentation.
b. List the memory management methods discussed, and indicate the types of fragmentation caused by each method.

Lecture Notes by Uğur Halıcı 29

EE442 Operating Systems Ch. 3 Memory Management

3. Consider a multiuser computer system which uses paging. The system has four associative registers. the content of these registers and page table for user_12 are given below:

Page table for user_12 associative registers

0 9 user # page # frame #

1 6 12 3 7

2 15 5 2 18

3 7 12 4 42

4 42 9 0 10

PTLR[12]:5 PTBR[12]:50000

PAGE SIZE :1024 words

For the following logical addresses generated by user_12's program, calculate the physical addresses, explain how those physical addresses are found, and state the number of physical memory accesses needed to find the corresponding word in memory. If a given logical address is invalid, explain the reason.

- i. <2,1256> ii. <3,290>
- iii. <4,572> iv. <5,290>
- v. <0,14>

4. The following memory map is given for a computer system with variable partitioning memory management.

0 Job Requested memory

J1 (50K)

1) J4 arrives 10K

free (45K) 2) J5 arrives 20K

3) J6 arrives 15K

J2 (40K) 4) J7 arrives 20K

free (10K) 5) J3 leaves

J3 (20K) 6) J8 arrives 50K

free (30K)

Find and draw the resulting memory maps, after each step of the above job sequence is processed, for :

a. first-fit b. best-fit c. worst-fit

5. Consider a computer system which uses paging. Assume that the system also has associative registers.

a. Explain how logical addresses are translated to physical addresses.

Lecture Notes by Uğur Halıcı 30

EE442 Operating Systems Ch. 3 Memory Management

b. Calculate the effective memory access time given:

assoc. register access time = 50 nanosec.

memory access time = 250 nanosec.

hit ratio = 80%

c. With the above associative register and memory access times, calculate the minimum hit ratio to give an effective memory access time less than 320 nanoseconds.

6. A system which utilizes segmentation gives you the ability to share segments. If the segment numbers are fixed, then a shared segment table is needed, and the ST must be modified. We shall assume that the system is capable of dynamic relocation, but to reduce the overhead, we want to avoid it unless it is absolutely necessary. The following example is given for such a system :

ST-6 ST-9

s# base size shares s# base size shares

0 - - 256 0 190 100 -

1 0 100 - 1 - - 256

2 100 90 - 2 290 10 -

3 600 15 -

SST

s# base size no. of sh.

256 400 200 2

Assume maximum number of segments per process is 256, and segments are numbered starting with 0.

- What would be done when a segment previously unshared, becomes a shared segment?
- When do we need dynamic relocation in this system?
- Assume segment-2 of process 6 is being executed. A reference is made to segment-0 of process 6. How is the corresponding physical address going to be found?
- How would self-references within shared segments be handled?
- What is the no. of sharers field in SST used for?

7. In the X-2700 computer, logical addresses are 24 bits long. The machine implements paged segmentation with a maximum segment size of 64K words and 4K-word pages:

- Show the logical address structure indicating the segment, page and displacement bits.
- How many segments can a user process contain?
- If a process has to be fully loaded into memory to execute, what is the minimum physical memory capacity?
- If the memory unit contains 65536 frames, show the physical address structure.

Lecture Notes by Uğur Halıcı 31

EE442 Operating Systems Ch. 3 Memory Management

e. Show the functional block structure of a suitable architecture for implementing paged segmentation in the X-2700. Indicate the sizes of all necessary tables.

8. Given the memory map in the figure, where areas not shaded indicate free regions, assume that the following events occur:

Step event required contiguous P1

memory size (K)

----- <free> 30 K

i) process 5 arrives 16

P2

ii) process 6 arrives 40

iii) process 7 arrives 20 <free> 20 K

iv) process 8 arrives 14

v) process 5 leaves - P3

vi) process 9 arrives 30

<free> 50 K

P4

a. Draw the memory maps after step (iv) and (vi) using first fit, best-fit and worst-fit allocation techniques, without compaction

- b. Draw the same memory maps as in part (a) if compaction is performed whenever required. Also show the maps after each compaction.
9. In a paging system, the logical address is formed of 20 bits. the most significant 8 bits denote the page number, the least significant 12 bits denote the offset. Memory size is 256K bits.
- What is the page size (in bits)?
 - What is the maximum number of pages per process?
 - How many frames does this system have?
 - Give the structure of the page table of a process executing in this system. Assume 2 bits are reserved for attributes.
 - How many bits are required for each page table entry?
 - If physical memory is upgraded to 1024 K bits, how will your answer to c and e change?
10. Consider a segmentation system with the following data given:

STBR=1000

STLR=5

Associative Registers access time = 50 nsec

Memory Access time = 500 nsec

Lecture Notes by Uğur Halıcı 32

EE442 Operating Systems Ch. 3 Memory Management

ST AR

s# base limit s# base limit

0 10000 1000 0 10000 1000

1 12000 2000 1 12000 2000

2 25000 4000

3 15000 8000

4 38000 4000

Assume data can be written into associative registers in parallel with a memory read or write operation. For replacement of data in associative registers, LRU policy is used.

For each of the following logical addresses, find the corresponding physical address to be accessed, and the total execution time required for that access, including the time spent for address translation operations. Also indicate which memory locations are accessed during address translation. Clearly show all changes made in associative registers.

- <0,150>
- <0,3700>
- <2,900>
- <2,3780>
- <5,200>
- <1,200>

11.

P1 Consider the memory map given in the figure. If worst fit policy is to be applied, then what will be the memory map
<free> 9K after arrival of the processes

P2

P5=3K, P6=5K, P7=7K P8=6K.

<free> 20 K

Indicate if compaction is needed.

P3

<free> 14 K

P4

12. The following memory map is given for a computer system with variable partitioning memory management.

P1 9K event required contiguous memory size (K)

i) P4 arrives 16

<free> 20 K ii) P5 arrives 40

iii) P6 arrives 20

P2 11K

iv) P7 arrives 14

<free> 10 K

Find and draw the resulting memory maps after the above job

P3 18K sequence is processed completely for

<free> 30 K a. first fit b. best fit c. worst fit

indicating whenever a compaction is needed.

Lecture Notes by Uğur Halıcı 33